UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

A.A 2025/2026

Tesi di laurea magistrale

Scheduling adattivo carbon-aware in ambienti serverless: gestione del trade-off energia-accuratezza tramite varianti di funzione

RELATORI:
Prof.ssa Valeria Cardellini
Prof. Gabriele Russo Russo

CANDIDATO:

Lorenzo Grande
0350212

Indice

Abstract **4**

Ringraziamenti **5**

1 Introduzione **6**

1.1 Contesto e motivazioni 6

1.1.1 Le criticità energetiche dell'esecuzione all'edge 7

1.1.2 Il problema della granularità nella misurazione energetica 8

1.1.3 Approximate Computing come leva per la riduzione dei consumi 9

1.1.4 Verso uno scheduling energy-aware con consenso esplicito 9

1.2 Contributi e obiettivi della tesi 10

1.3 Struttura della tesi 11

2 Stato dell'arte **13**

2.1 Il monitoraggio dell'energia software 13

2.1.1 Misuratori hardware del consumo energetico 15

2.1.2 Misuratori software e modelli di stima 19

2.2 Elaborazione serverless sostenibile 23

2.2.1 Scheduling energeticamente consapevole nelle piattaforme FaaS 24

2.2.1.1 Scheduling basato sullo stato energetico dei nodi . . . 26
2.2.1.2 Scheduling per consolidamento energetico con vincoli di deadline . . . 28
2.2.1.3 Scheduling Energy-Aware in Ambienti FaaS Federati ed Eterogenei . . . 30
2.2.2 Power capping come tecnica di controllo energetico . . . 33
2.2.2.1 Applicabilità degli approcci tradizionali nel contesto serverless . . . 33
2.2.2.2 Impatto sulle prestazioni e approcci adattivi . . . 34
2.2.3 Direzioni emergenti per l'elaborazione serverless sostenibile . . . 36
2.3 Il paradigma FaaS in ambienti cloud-edge . . . 38
2.3.1 La piattaforma Serverledge . . . 39
2.4 Posizionamento del contributo . . . 40

3 Progettazione e Realizzazione 42

3.1 Integrazione di Kepler per il monitoraggio energetico . . . 42
3.1.1 Funzionamento di Kepler . . . 43
3.1.2 Infrastruttura di supporto . . . 44
3.2 Il modello di funzione logica con varianti . . . 46
3.2.1 Struttura della funzione e profilo energetico . . . 46
3.2.1.1 Determinazione dei profili energetici iniziali . . . 47
3.2.2 Il modello di output e la classificazione dell'accuratezza . . . 49
3.2.3 Registrazione e indicizzazione delle varianti in etcd . . . 51
3.2.3.1 Convenzione di denominazione . . . 52
3.2.3.2 Struttura delle chiavi in etcd . . . 52

3.3 Misurazione e aggiornamento adattivo dei profili energetici 54
3.3.1 Tracciamento dei container e conteggio delle invocazioni 54
3.3.2 Il worker periodico di raccolta 56
3.3.3 Attribuzione dell'energia per invocazione e gestione del lag . . . 57
3.3.4 Aggiornamento del profilo tramite media mobile esponenziale . . 57
3.3.5 Archiviazione su InfluxDB 58
3.4 Lo scheduler energy-aware 59
3.4.1 Il consenso esplicito dell'utente 60
3.4.2 Selezione per errore minimo 61
3.4.3 Selezione con vincolo energetico 63

4 Risultati Sperimentali 67

5 Conclusioni 68

Elenco delle figure 69

Elenco dei codici 71

Elenco degli algoritmi 72

Abstract

Ringraziamenti

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Negli ultimi anni si è assistito a una progressiva trasformazione dell'architettura dei sistemi distribuiti: servizi e applicazioni, tradizionalmente eseguiti in data center centralizzati, vengono sempre più frequentemente spostati verso il bordo della rete. Questa tendenza, nota come *edge computing*, nasce dalla necessità di ridurre la latenza percepita dagli utenti, supportare applicazioni real-time e limitare il traffico verso il cloud [1]. Il paradigma emergente non è più quello di un cloud monolitico, ma di un *continuum Edge–Cloud*, in cui risorse computazionali eterogenee e geograficamente distribuite collaborano per soddisfare i requisiti applicativi. In tale contesto, l'elaborazione può essere collocata dinamicamente lungo la catena edge–cloud in funzione delle condizioni di rete, della disponibilità di risorse e dei vincoli applicativi.

Serverless Computing e FaaS nel continuum Edge–Cloud

Parallelamente all'evoluzione verso l'edge, il paradigma *Serverless Computing* ha acquisito crescente rilevanza come modello di programmazione cloud-native. Nel modello serverless, lo sviluppatore distribuisce codice senza occuparsi della gestione dell'infrastruttura sottostante, che viene completamente delegata al provider [2]. Il modello di

fatturazione *pay-as-you-go* e la scalabilità automatica hanno contribuito alla sua ampia adozione nell'industria.

All'interno dell'ecosistema serverless, il modello *Function-as-a-Service* (FaaS) rappresenta la forma più diffusa di esecuzione event-driven. In una piattaforma FaaS, l'unità fondamentale di esecuzione è una funzione stateless, invocata in risposta a eventi o richieste e tipicamente eseguita all'interno di container o ambienti isolati equivalenti.

L'estensione del paradigma FaaS al continuum Edge–Cloud risulta particolarmente promettente: eseguire funzioni direttamente su nodi edge consente di ridurre la latenza di risposta e migliorare la qualità del servizio percepita. Recenti lavori hanno mostrato come architetture FaaS decentralizzate possano abilitare l'esecuzione distribuita di funzioni tra edge e cloud, introducendo meccanismi di offloading verticale e orizzontale [3].

1.1.1 Le criticità energetiche dell'esecuzione all'edge

Nonostante i benefici in termini di latenza e prossimità ai dati, l'esecuzione all'edge introduce criticità significative. I nodi edge sono tipicamente caratterizzati da risorse computazionali limitate, eterogeneità hardware e condizioni operative variabili. A differenza dei data center cloud, progettati per offrire elevata capacità e ridondanza, i nodi edge devono operare con vincoli stringenti su CPU, memoria e, soprattutto, energia disponibile.

In molti scenari applicativi – dispositivi alimentati a batteria, infrastrutture soggette a limiti di potenza, contesti IoT – l'energia non rappresenta soltanto un costo operativo, ma una risorsa limitata e dinamica. La letteratura sul resource scheduling in ambienti edge evidenzia come la gestione efficiente delle risorse, inclusa l'energia, costituisca una delle principali sfide ancora aperte [4, 5].

In tali contesti non è sufficiente adottare politiche che minimizzano il consumo medio di energia: diventa necessario controllare l'energia associata alle singole esecuzioni, garantendo il rispetto di budget energetici prefissati e potenzialmente variabili nel tempo.

1.1.2 Il problema della granularità nella misurazione energetica

Un prerequisito fondamentale per qualsiasi meccanismo di controllo energetico è la disponibilità di misure affidabili e aggiornate a granularità sufficientemente fine. Storicamente, la stima del consumo energetico nei sistemi virtualizzati è stata affrontata a livello di macchina fisica o di macchina virtuale, rendendo difficile attribuire il costo energetico a singoli componenti applicativi.

Nel contesto FaaS, questa difficoltà è amplificata dalla natura *fine-grained* delle invocazioni e dalla variabilità delle condizioni runtime, quali il carico concorrente, le interferenze tra container e la distinzione tra esecuzioni a freddo (*cold start*) e a caldo (*warm start*). Negli ultimi anni, strumenti di osservabilità energetica per ambienti containerizzati hanno reso disponibile una misurazione a granularità significativamente più fine.

In particolare, Kepler (*Kubernetes-based Efficient Power Level Exporter*) stima il consumo energetico a livello di singolo processo e container in ambienti Kubernetes, esportando metriche accessibili in tempo reale tramite Prometheus [6]. La disponibilità di misurazioni energetiche a livello di container costituisce un abilitatore fondamentale per integrare l'energia tra le metriche decisionali delle piattaforme FaaS, aprendo la strada a meccanismi di scheduling energeticamente consapevoli.

1.1.3 Approximate Computing come leva per la riduzione dei consumi

Una direzione promettente per la riduzione del consumo energetico in ambito FaaS è rappresentata dal paradigma dell'*Approximate Computing*, che consente di ridurre il costo computazionale accettando, in modo controllato, una diminuzione dell'accuratezza del risultato prodotto. Lavori come JouleGuard [7] hanno mostrato come sia possibile progettare meccanismi di controllo runtime capaci di rispettare un budget energetico prefissato, regolando dinamicamente il livello di approssimazione dell'esecuzione.

Applicata al paradigma FaaS, questa idea si traduce nella possibilità di associare a una stessa funzione logica più varianti implementative, caratterizzate da diversi profili energetici e da diversi livelli di accuratezza del risultato. La selezione della variante da eseguire può quindi diventare una decisione dello scheduler, guidata da informazioni energetiche aggiornate a runtime e dai vincoli espressi dall'utente al momento dell'invocazione.

1.1.4 Verso uno scheduling energy-aware con consenso esplicito

La combinazione tra (i) l'adozione crescente di architetture edge–cloud ibride, (ii) i vincoli energetici stringenti nei nodi edge e (iii) la disponibilità di misurazioni energetiche a granularità container-level rende oggi possibile e necessario studiare meccanismi di esecuzione in grado di controllare dinamicamente l'energia associata alle funzioni serverless.

Tuttavia, la riduzione del consumo energetico ottenuta tramite approssimazione introduce un compromesso con l'accuratezza del risultato che non può essere gestito unilateralmente dalla piattaforma. Tale compromesso richiede un *consenso esplicito* da parte dell'utente che invoca la funzione: solo in presenza di tale consenso lo scheduler

è autorizzato a selezionare varianti approssimate. Questa scelta progettuale garantisce che il comportamento del sistema resti trasparente e controllabile, distinguendo nettamente tra ottimizzazione energetica e degradazione indesiderata della qualità.

1.2 Contributi e obiettivi della tesi

L'obiettivo principale di questo lavoro non è la mera riduzione del consumo energetico medio di una piattaforma serverless, bensì la progettazione e realizzazione di un'estensione energy-aware della piattaforma Serverledge [3] in grado di operare sotto vincoli energetici espliciti e di adattare le decisioni di esecuzione in funzione di informazioni energetiche aggiornate a runtime.

In particolare, il lavoro si propone di:

- introdurre il concetto di *funzione logica* associata a un insieme di varianti implementative alternative, ciascuna descritta da un profilo energetico che ne quantifica il costo atteso per invocazione, il costo di cold start e, ove applicabile, la stima dell'errore introdotto dall'approssimazione;
- integrare un sistema di monitoraggio energetico a livello di container basato su Kepler, strumento che stima il consumo energetico a granularità di processo nei nodi Kubernetes e lo esporta tramite Prometheus, consentendo la raccolta di misure energetiche reali durante l'esecuzione delle funzioni;
- progettare e implementare un meccanismo di aggiornamento dinamico dei profili energetici delle varianti, realizzato tramite un worker periodico che legge le misure cumulative da Kepler, calcola il consumo per singola invocazione e aggiorna i profili archiviati in etcd, realizzando un ciclo di feedback adattivo;

- progettare e implementare uno scheduler energeticamente consapevole, ispirato concettualmente ai meccanismi di controllo basati su budget energetico proposti in letteratura, che al momento dell'invocazione seleziona la variante più appropriata in funzione di:
  - il budget energetico massimo espresso dall'utente (`MaxEnergyJoule`);
  - il consenso esplicito dell'utente all'utilizzo di varianti approssimate (`AllowApprox`);
  - le stime energetiche aggiornate disponibili a runtime;

  rispettando il vincolo che, in assenza di consenso esplicito, vengano selezionate esclusivamente varianti esatte;
- valutare sperimentalmente il compromesso tra consumo energetico e accuratezza del risultato su un insieme di funzioni rappresentative, dimostrando la fattibilità e l'efficacia di un modello di esecuzione adattivo per il paradigma Function-as-a-Service nel continuum Edge–Cloud.

Il contributo complessivo della tesi consiste nell'estensione di una piattaforma FaaS open-source con un meccanismo di esecuzione adattivo rispetto a vincoli energetici dinamici, introducendo un compromesso controllato tra energia consumata e accuratezza del risultato, condizionato al consenso esplicito dell'utente che effettua l'invocazione.

1.3 Struttura della tesi

Il **Capitolo 2** analizza lo stato dell'arte relativo all'elaborazione serverless sostenibile, esaminando le principali soluzioni proposte in letteratura per il monitoraggio energetico del software, con particolare attenzione agli strumenti capaci di operare a granularità

container-level. Vengono inoltre discusse le tecniche di scheduling energy-aware per ambienti FaaS e le direzioni emergenti nell'ambito dell'Approximate Computing applicato all'esecuzione serverless. Il capitolo si conclude con la presentazione della piattaforma Serverledge, adottata come base sperimentale per lo sviluppo del sistema proposto.

Il **Capitolo 3** descrive le scelte progettuali e i meccanismi realizzati. Vengono presentati il modello di funzione logica con varianti energetiche, l'integrazione di Kepler per il monitoraggio dei consumi a livello di container, il worker periodico responsabile dell'aggiornamento adattivo dei profili energetici e la logica decisionale dello scheduler energy-aware, con particolare attenzione al ruolo del consenso esplicito dell'utente nella selezione di varianti approssimate.

Il **Capitolo 4** riporta i risultati della valutazione sperimentale condotta su un insieme di funzioni di test rappresentative di differenti profili computazionali. Vengono analizzati il comportamento dello scheduler al variare del budget energetico, il risparmio energetico ottenuto tramite le varianti approssimate e il corrispondente impatto sull'accuratezza del risultato, quantificando il compromesso introdotto dalle scelte progettuali adottate.

Il **Capitolo 5** raccoglie le conclusioni del lavoro, discutendo i risultati ottenuti in relazione agli obiettivi definiti e identificando le principali direzioni di sviluppo futuro.

.

Capitolo 2

Stato dell'arte

Il capitolo analizza le principali soluzioni di monitoraggio del consumo energetico e le tecniche di risparmio energetico applicate al Serverless Computing, mettendo in evidenza limiti e ambiti di applicazione. L'attenzione è posta in particolare sui problemi legati alla stima del consumo energetico in contesti dinamici e distribuiti, dove le metriche disponibili risultano spesso statiche o ottenute offline. Viene quindi evidenziata la necessità di disporre di metriche energetiche aggiornate nel tempo, in grado di riflettere il comportamento reale delle esecuzioni. Tali metriche rappresentano un elemento chiave per supportare politiche di scheduling energy-aware soggette a vincoli energetici espliciti. Questo capitolo inquadra il contributo della tesi nel contesto delle soluzioni esistenti

2.1 Il monitoraggio dell'energia software

Il monitoraggio del consumo energetico a livello software rappresenta un tema di ricerca centrale nell'ambito dell'efficienza energetica dei sistemi informatici moderni. La crescente complessità delle architetture hardware e la diffusione di modelli di calcolo dinamici e distribuiti hanno reso sempre più difficile ottenere misure energetiche accurate e direttamente attribuibili alle singole componenti software. Questo problema

risulta particolarmente rilevante nel contesto del Serverless Computing, dove le unità di esecuzione sono caratterizzate da invocazioni brevi, stateless e altamente dinamiche.

Le soluzioni tradizionali di monitoraggio dell'energia si basano prevalentemente su misurazioni a livello hardware, che forniscono una stima del consumo complessivo di una macchina fisica o di specifici componenti, come CPU e memoria. Sebbene tali approcci offrano un'elevata precisione, la loro granularità risulta spesso insufficiente per analizzare il comportamento energetico delle applicazioni software, soprattutto in ambienti multi-tenant e containerizzati [8]. In questi scenari, il consumo energetico osservato è il risultato dell'interazione di più entità software concorrenti, rendendo complessa l'attribuzione dei costi energetici alle singole esecuzioni.

Per superare queste limitazioni, negli ultimi anni sono state proposte numerose soluzioni di monitoraggio dell'energia software, che mirano a stimare il consumo energetico a livelli di granularità più fini, quali processo, container o applicazione. Tali approcci si basano generalmente su modelli di stima che correlano metriche osservabili del sistema con misure energetiche ottenute a livello hardware, come nel caso di PowerAPI e SmartWatts [9].

Nel contesto del Serverless Computing, il problema del monitoraggio energetico è ulteriormente aggravato dalla crescente diffusione di architetture cloud-edge. Per ridurre la latenza e migliorare la qualità del servizio, sempre più spesso la computazione viene spostata verso nodi edge prossimi alla sorgente dei dati, anziché su data center remoti [1]. Come già accennato, tali nodi sono frequentemente caratterizzati da risorse limitate e, in molti casi, sono alimentati a batteria o da fonti di energia intermittenti. In questi scenari, l'energia disponibile diventa una risorsa critica e vincolante.

Le condizioni operative dei nodi edge possono variare nel tempo in funzione dello stato di carica della batteria, del carico di lavoro e delle caratteristiche dell'hardware

sottostante. Di conseguenza, metriche energetiche statiche o ottenute tramite profilazione offline risultano spesso inadeguate a rappresentare il comportamento reale delle esecuzioni. Diventa quindi fondamentale disporre di metriche energetiche aggiornate nel tempo, in grado di supportare decisioni di esecuzione soggette a vincoli energetici espliciti, ponendo le basi per politiche di scheduling energy-aware.

2.1.1 Misuratori hardware del consumo energetico

I misuratori hardware del consumo energetico rappresentano l'approccio più diretto per la misurazione dell'energia assorbita da un sistema di calcolo. Tali soluzioni si basano su sensori fisici, esterni o integrati nell'hardware, e forniscono misure caratterizzate da un'elevata accuratezza, in quanto l'energia viene misurata direttamente a livello fisico. Il monitoraggio può avvenire a livello di intera macchina o di specifici componenti hardware, come CPU e memoria.

Una prima categoria è rappresentata dai misuratori hardware esterni, ovvero dispositivi interposti fisicamente tra la sorgente di alimentazione e il sistema sotto analisi. Questi strumenti consentono di misurare l'energia realmente dissipata dall'intero ecosistema hardware, includendo componenti spesso non osservabili tramite sensori software, come gli alimentatori (PSU), i sistemi di raffreddamento e le perdite resistive dei circuiti. Per tale motivo, essi forniscono una visione completa del consumo energetico del sistema.

Tuttavia, come ampiamente documentato in letteratura [10], l'adozione di misuratori hardware esterni introduce una serie di criticità che ne limitano l'applicabilità in contesti operativi su larga scala. In primo luogo, la misurazione avviene tipicamente a livello di intero sistema (*full-system power*), rendendo complesso isolare il contributo energetico di singoli componenti hardware o di specifiche entità software senza ricorrere

Tabella 2.1: Confronto tra principali tipologie di misuratori hardware esterni per il monitoraggio del consumo energetico.

Categoria	**Esempio di Dispositivo**	**Costo Stimato**	**Ambito di utilizzo**
Entry-level	TP-Link Kasa, Kill-A-Watt	€30 – €60	Analisi preliminari e misure indicative su singole workstation.
Enterprise	APC Metered Rack PDU, Raritan PX	€800 – €2.500	Monitoraggio energetico di rack o gruppi di server in data center.
Laboratorio / R&D	Yokogawa WT310E, ZES ZIMMER LMG	€2.500 – €6.000+	Misurazioni ad alta precisione per validazione scientifica e ricerca accademica.

a strumentazione aggiuntiva e altamente invasiva. In secondo luogo, l'integrazione di tali dispositivi in infrastrutture distribuite o in ambienti di data center comporta costi economici e complessità operative significative, legate sia all'acquisizione dell'hardware sia alla gestione del cablaggio e della raccolta dei dati.

Per fornire un ordine di grandezza dell'investimento richiesto, nella Tabella 2.1 sono riportate alcune delle principali categorie di strumenti utilizzati per il monitoraggio energetico hardware, insieme ai relativi ambiti di applicazione.

I dispositivi di fascia entry-level offrono generalmente frequenze di campionamento limitate, dell'ordine di pochi Hertz, risultando adeguati per analisi preliminari su singole macchine. Al contrario, gli strumenti utilizzati in ambito di ricerca e validazione scientifica consentono misurazioni ad alta frequenza e un'elevata precisione, a fronte di costi significativamente più elevati.

Oltre ai misuratori hardware esterni, i moderni processori integrano sensori energetici direttamente a livello di chip, rendendo possibile l'accesso a stime del consumo energetico tramite interfacce software. Un esempio significativo è rappresentato dalla tecnologia *Running Average Power Limit* (RAPL), introdotta da Intel, che consente

di ottenere informazioni sul consumo energetico di diversi domini di potenza interni al processore [11, 12].

RAPL suddivide il consumo energetico del sistema in un insieme di domini logici, ciascuno dei quali rappresenta una porzione specifica dell'hardware. In particolare, il dominio *Package* (PKG) rappresenta il consumo energetico complessivo del processore, includendo i core, le cache e le interconnessioni interne. Il dominio *Power Plane 0* (PP0) è invece associato principalmente ai core di calcolo della CPU, mentre altri domini, come *DRAM*, forniscono una stima del consumo energetico della memoria principale, quando supportati dall'architettura.

La Figura 2.1 illustra un esempio di suddivisione del consumo energetico secondo il modello RAPL, evidenziando come i diversi domini di potenza corrispondano a specifiche componenti hardware del sistema. Questa rappresentazione consente di comprendere come le misure fornite da RAPL siano intrinsecamente legate all'organizzazione fisica del processore e non alle singole entità software in esecuzione.

Il funzionamento di RAPL si basa su modelli energetici interni al processore, calibrati dal produttore, che stimano l'energia consumata a partire da eventi hardware e contatori interni. Sebbene tali misure non rappresentino una misurazione fisica diretta dell'energia, diversi studi hanno dimostrato come esse offrano un buon compromesso tra accuratezza e semplicità di utilizzo, risultando particolarmente adatte al monitoraggio energetico a runtime[13]. Le informazioni rese disponibili da RAPL possono essere lette tramite interfacce standard del sistema operativo, consentendo la loro integrazione in strumenti di monitoraggio software.

Nonostante questi vantaggi, i sensori hardware integrati presentano limiti strutturali quando applicati a contesti software moderni. La granularità delle misure è tipicamente limitata al livello di macchina o di componente hardware e non consente

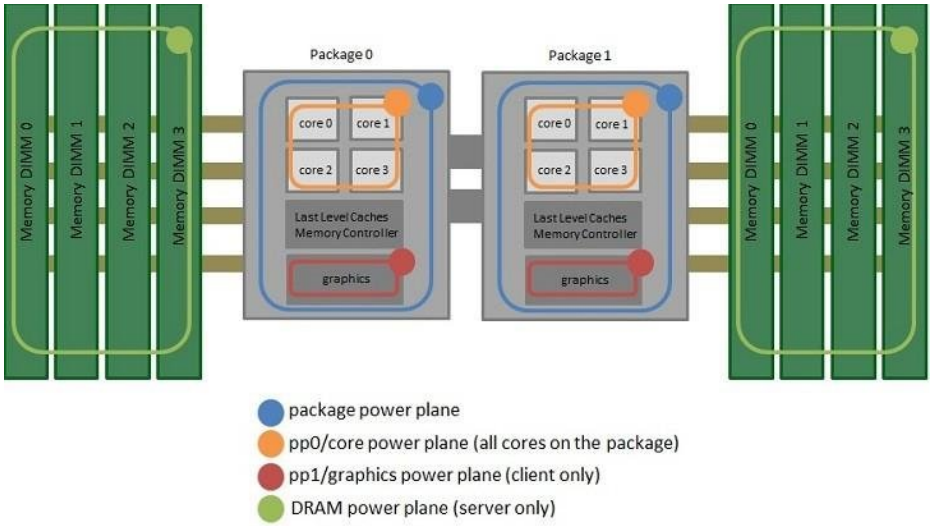

Figura 2.1: Esempio di domini di potenza definiti dal modello RAPL. Il dominio *Package* rappresenta il consumo energetico complessivo della CPU, mentre domini più specifici, come *PP0* e *DRAM*, forniscono stime riferite rispettivamente ai core di calcolo e alla memoria principale.

di attribuire in modo diretto il consumo energetico alle singole applicazioni, ai container o alle singole invocazioni. In ambienti multi-tenant e containerizzati, come quelli tipici del Serverless Computing, il consumo energetico osservato è quindi il risultato dell'esecuzione concorrente di più entità software, rendendo complessa l'analisi del comportamento energetico delle singole funzioni.

Inoltre, le misure fornite da RAPL sono di natura cumulativa e non distinguono in modo esplicito tra le diverse fasi di esecuzione, quali inizializzazione, esecuzione vera e propria e periodi di inattività. Questo aspetto risulta particolarmente critico nel Serverless Computing, dove fenomeni come il cold start e il riutilizzo dei container hanno un impatto significativo sul consumo energetico complessivo. Di conseguenza, pur costituendo una base affidabile per la stima dell'energia a livello di sistema, i sensori hardware integrati non risultano sufficienti per supportare analisi a grana fine e decisioni di gestione energetica a livello software.

Alla luce di queste considerazioni, sebbene i misuratori hardware costituiscano una base affidabile per la misurazione dell'energia, essi risultano insufficienti per supportare il monitoraggio continuo e a grana fine richiesto da ambienti dinamici e distribuiti. In tali contesti emerge la necessità di soluzioni in grado di offrire una maggiore flessibilità e di integrarsi in modo più stretto con i meccanismi di esecuzione del software.

2.1.2 Misuratori software e modelli di stima

Le limitazioni intrinseche dei misuratori hardware, quali la scarsa granularità temporale, i costi elevati di implementazione su larga scala e l'impossibilità di attribuire direttamente il consumo energetico a singole entità logiche in sistemi multi-tenant, hanno favorito l'emergere dei misuratori software. Questi strumenti non si basano su sensori fisici dedicati per ogni applicazione, ma utilizzano modelli di stima che correlano l'attività del software con il consumo energetico complessivo del sistema. Il principio fondamentale consiste nell'impiegare metriche di utilizzo delle risorse hardware — come cicli di clock della CPU, istruzioni eseguite, accessi alla cache e operazioni di input/output — come base per stimare l'energia assorbita dalle singole entità di esecuzione. Questo approccio consente di ottenere una visibilità energetica più granulare rispetto alle sole misure hardware, risultando particolarmente adatto a contesti moderni quali il cloud computing, le architetture a microservizi e i sistemi containerizzati.

Uno dei framework più rappresentativi in questo ambito è PowerAPI, una libreria modulare progettata per il monitoraggio del consumo energetico a livello di processo attraverso componenti indipendenti e meccanismi di raccolta dati asincroni e scalabili [9]. L'architettura di PowerAPI è progettata per separare nettamente la fase di raccolta delle metriche da quella di stima energetica, favorendo l'estendibilità e la portabilità del sistema. Il pilastro tecnologico di PowerAPI è rappresentato dall'**HWPC Sensor**

(*Hardware Performance Counter Sensor*), un componente specializzato nel campionamento ad alta frequenza dei contatori di prestazione hardware [14]. Il sensore si interfaccia direttamente con il sottosistema `perf_event` del kernel Linux, permettendo di raccogliere metriche dettagliate sull'attività della CPU associate a specifici *Control Groups* (cgroups) [1] In parallelo, l'HWPC Sensor interroga i *Model Specific Registers* (MSR) tramite l'interfaccia *Running Average Power Limit* (RAPL), ottenendo misure cumulative del consumo energetico dei principali domini della CPU [13].

La Figura 2.2 mostra una rappresentazione concettuale dell'architettura di PowerAPI. Le metriche di utilizzo delle risorse e le informazioni energetiche fornite dai sensori hardware vengono elaborate dal modello di stima **SmartWatts**, adottato da PowerAPI per attribuire il consumo energetico ai singoli processi.

SmartWatts utilizza tecniche di regressione lineare per scomporre il consumo energetico globale in una componente statica e una dinamica, consentendo un'attribuzione più accurata rispetto a modelli puramente statici [15]. Tale approccio richiede una fase di auto-calibrazione *online* e l'accesso diretto all'hardware sottostante.

Tali requisiti rendono PowerAPI particolarmente adatto a scenari *bare-metal*, ma ne limitano l'applicabilità in ambienti virtualizzati e cloud pubblici, dove l'accesso ai registri hardware della CPU e ai performance counter è spesso inibito dagli hypervisor per motivi di sicurezza e isolamento. In questi contesti emerge la necessità di strumenti di monitoraggio energetico progettati nativamente per ambienti orchestrati e multi-tenant.

Un approccio rappresentativo in questa direzione è **Kepler** (*Kubernetes-based Effi-*

[1] Il sottosistema `perf_event` del kernel Linux fornisce un'interfaccia unificata per il monitoraggio di eventi hardware e software a basso livello, come cicli di clock della CPU, istruzioni eseguite e cache miss. Attraverso la chiamata di sistema `perf_event_open()`, consente di associare le misurazioni a processi, thread o *cgroups*, esponendo i dati tramite *file descriptor* e buffer ad anello con overhead contenuto. `perf_event` costituisce inoltre la base del tool `perf`, ampiamente utilizzato per la profilazione delle prestazioni nei sistemi Linux.

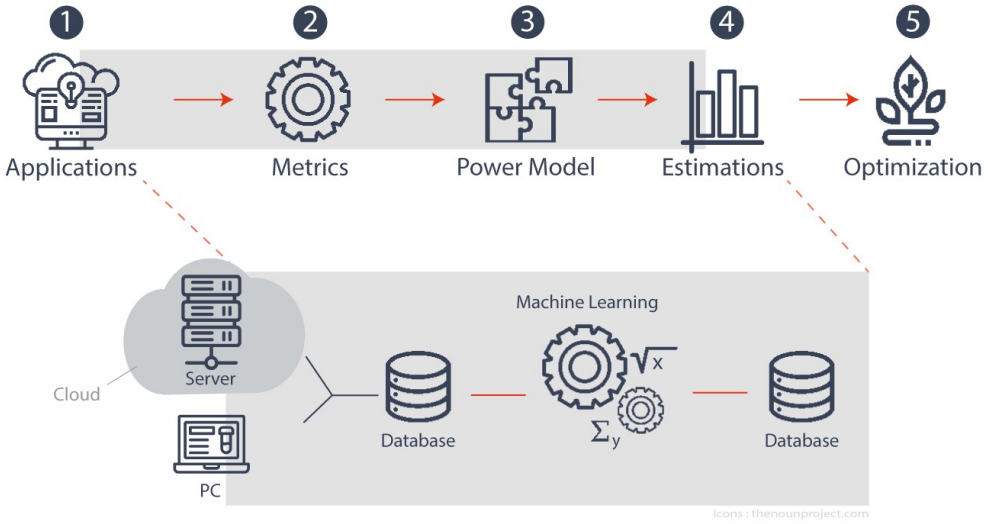

Figura 2.2: Architettura concettuale di PowerAPI. I contatori di prestazione hardware e le misure energetiche aggregate (RAPL) vengono combinati dal modello SmartWatts per stimare il consumo energetico dei singoli processi.

cient Power Level Exporter), un progetto *cloud-native* orientato al monitoraggio energetico di container e Pod in cluster Kubernetes [16, 17]. A differenza di PowerAPI, Kepler non richiede accesso diretto ai registri hardware della CPU, ma sfrutta la tecnologia *eBPF* (*Extended Berkeley Packet Filter*) per raccogliere metriche a livello kernel in modo non intrusivo e con overhead computazionale ridotto.

La Figura 2.3 mostra l'architettura complessiva di Kepler. Il sistema utilizza un *eBPF Program Generator* per generare dinamicamente programmi eBPF che vengono agganciati ai *tracepoint* del kernel e ai performance counter della CPU. Questi programmi intercettano eventi di schedulazione e statistiche di esecuzione, consentendo di associare l'attività computazionale ai singoli container in modo fine-grained.

Le informazioni raccolte tramite eBPF vengono arricchite dal *Pod Lister*, che interroga le API di Kubernetes per mappare gli identificativi dei container ai rispettivi Pod e raccoglie statistiche aggiuntive dai *control groups*. Parallelamente, il componente

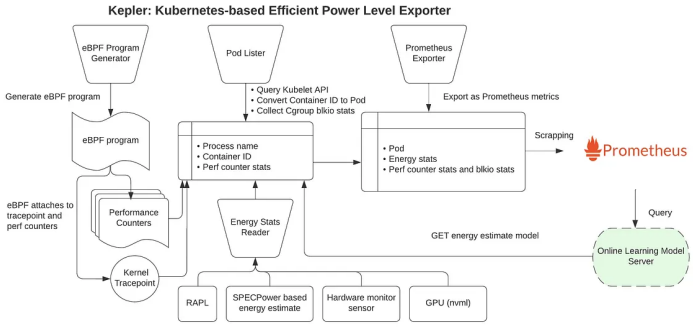

Figura 2.3: Architettura di Kepler.

Energy Stats Reader acquisisce informazioni energetiche aggregate dal nodo, utilizzando diverse fonti a seconda della disponibilità dell'hardware sottostante, tra cui RAPL, sensori hardware esterni, interfacce GPU (NVML) e modelli basati su benchmark standardizzati.

Quando le misure hardware dirette non risultano accessibili, scenario tipico delle macchine virtuali nei cloud pubblici, Kepler ricorre a modelli di stima basati su *Machine Learning*, denominati *Estimators*. Tali modelli, addestrati offline su differenti architetture hardware e carichi di lavoro, ricevono in input le metriche raccolte a livello kernel e producono una stima della potenza assorbita dai container, che viene successivamente integrata nel tempo per ottenere una stima energetica complessiva.

Le metriche prodotte da Kepler vengono infine esposte tramite un *Prometheus Exporter*, rendendo il consumo energetico direttamente interrogabile attraverso i meccanismi di *scraping* standard e facilmente integrabile con strumenti di visualizzazione. Questa integrazione nativa con l'ecosistema di osservabilità cloud consente di affiancare le metriche energetiche a quelle di performance, abilitando analisi congiunte e politiche di gestione consapevoli dal punto di vista energetico.

2.2 Elaborazione serverless sostenibile

Nel paradigma serverless, e in particolare nelle piattaforme Function-as-a-Service (FaaS), il consumo energetico è strettamente determinato dalla natura del modello di esecuzione: le funzioni sono entità effimere, stateless e isolate in ambienti containerizzati, invocate su richiesta con cicli di vita che si misurano spesso in millisecondi. Tale granularità, combinata con l'elevata variabilità del carico di lavoro, rende la gestione dell'energia una dimensione strutturalmente più complessa rispetto ai contesti HPC o ai data center tradizionali, nei quali le tecniche consolidate operano tipicamente a livello di nodo fisico e su scale temporali più ampie.

Allo stato dell'arte, l'efficienza energetica in ambienti serverless è affrontata prevalentemente attraverso interventi a livello di piattaforma, generalmente trasparenti per l'utente. Le soluzioni più diffuse agiscono su tre leve principali: la pianificazione energeticamente consapevole delle funzioni (*energy-aware scheduling*), la limitazione dinamica della potenza erogata alle risorse (*power capping*) e la riduzione dell'impatto energetico associato ai tempi di inizializzazione degli ambienti di esecuzione (*cold start overhead*).

In questi approcci, il consumo energetico viene trattato come una metrica aggiuntiva nel processo decisionale, tipicamente combinata con indicatori di prestazione o di qualità del servizio. L'obiettivo è la riduzione del consumo medio o complessivo del sistema, senza che vengano introdotti vincoli energetici espliciti o meccanismi di garanzia formale sul rispetto di un determinato budget. Questa impostazione, sebbene efficace in scenari con risorse abbondanti, mostra limiti evidenti in contesti caratterizzati da risorse limitate o da stringenti requisiti operativi — tra cui ambienti edge — nei quali il consumo energetico non può essere trattato come una variabile da ottimizzare in senso

best-effort, ma deve costituire un vincolo esplicito nel processo di pianificazione.

Una direzione complementare, ancora poco esplorata nel contesto serverless, consiste nell'agire sulla funzione stessa piuttosto che sull'infrastruttura: attraverso il paradigma dell'*Approximate Computing*, è possibile generare varianti della medesima funzione che riducono il consumo energetico accettando, in modo controllato, una diminuzione dell'accuratezza del risultato. Tale approccio sposta il problema dalla gestione delle risorse alla progettazione della logica applicativa, aprendo la strada a modelli di controllo runtime in grado di selezionare dinamicamente il livello di approssimazione più adeguato in funzione del budget energetico disponibile.

Nei paragrafi successivi vengono analizzati i principali contributi proposti in letteratura, seguendo una progressione che va dagli interventi infrastrutturali — scheduling energy-aware e power capping — verso meccanismi di controllo applicativo basati su vincoli energetici espliciti, delineando il contesto nel quale si inserisce il contributo della presente tesi.

2.2.1 Scheduling energeticamente consapevole nelle piattaforme FaaS

Nel paradigma Function-as-a-Service (FaaS), lo scheduling non costituisce soltanto un meccanismo di allocazione delle risorse, ma rappresenta uno strumento strategico per il controllo del consumo energetico dell'infrastruttura. A differenza delle politiche di pianificazione tradizionali, orientate prevalentemente all'ottimizzazione delle prestazioni o al bilanciamento del carico, lo scheduling energeticamente consapevole integra il consumo energetico tra i criteri che guidano le decisioni di *placement* e la selezione delle risorse di esecuzione.

Nel contesto serverless, tale problematica presenta caratteristiche peculiari legate

al modello di esecuzione. La breve durata delle funzioni, la loro natura stateless e l'isolamento in ambienti containerizzati determinano un'elevata frequenza delle decisioni di scheduling e una forte sensibilità alle condizioni operative. In particolare, l'inizializzazione di nuovi ambienti di esecuzione (*cold start*) può generare picchi di consumo energetico istantaneo che influenzano in modo non trascurabile l'efficienza complessiva del sistema. Per questa ragione, la gestione del riutilizzo degli ambienti di esecuzione (*warm scheduling*) rappresenta una dimensione rilevante tanto per la latenza quanto per il profilo energetico.

L'analisi allo stato dell'arte mostra come l'integrazione della dimensione energetica nel processo decisionale possa assumere forme concettualmente distinte, in funzione delle assunzioni architetturali e degli obiettivi perseguiti. Una prima classe di approcci considera il consumo energetico come una metrica aggiuntiva in modelli di ottimizzazione multi-obiettivo, con l'intento di ridurre il consumo medio del sistema senza introdurre vincoli espliciti. Una seconda classe tratta l'energia come vincolo infrastrutturale, adottando politiche di redistribuzione del carico in base alla disponibilità energetica corrente dei nodi. Una terza direzione, rilevante in scenari distribuiti e federati, estende la consapevolezza energetica al costo di comunicazione, incorporando nella decisione di *placement* il costo energetico della movimentazione dei dati tra siti eterogenei.

Nei paragrafi seguenti vengono analizzati tre contributi rappresentativi di queste direzioni. Faashouse [18] esemplifica l'approccio basato sullo stato energetico dei nodi, con redistribuzione dinamica del carico in ambienti edge alimentati da fonti intermittenti. E^2FIS [19] rappresenta invece una strategia di consolidamento del carico orientata alla minimizzazione del numero di nodi attivi, con gestione esplicita delle deadline. GreenFaaS [20] estende infine la prospettiva agli ambienti federati multi-sito, integran-

do nella funzione di scheduling una stima del costo energetico dei trasferimenti tra domini amministrativi distinti.

2.2.1.1 Scheduling basato sullo stato energetico dei nodi

Una prima tipologia di scheduler energeticamente consapevoli considera l'energia come proprietà dei nodi dell'infrastruttura, utilizzando lo stato energetico corrente come criterio primario per le decisioni di *placement*. In questo paradigma, le invocazioni vengono redistribuite tra nodi con diversa disponibilità energetica al fine di preservare la continuità operativa dell'infrastruttura nel suo complesso.

Tale approccio è particolarmente rilevante in scenari di Edge Computing distribuito, nei quali i nodi possono essere alimentati da fonti rinnovabili intermittenti e presentare livelli di carica variabili nel tempo. In questi contesti, l'obiettivo dello scheduler non è la minimizzazione del consumo medio, bensì il mantenimento dell'operatività dell'intero sistema, evitando che nodi con disponibilità energetica ridotta diventino temporaneamente irraggiungibili.

Un esempio rappresentativo di questo approccio è il sistema *Faashouse* [18], progettato per ambienti serverless all'estremo edge. Come illustrato in Figura 2.4, l'architettura si articola su più livelli: dispositivi edge a bassa potenza, piattaforma di containerizzazione e orchestrazione, e livello serverless per la gestione delle funzioni.

Il *Resource Scheduler*, ospitato nel controller, implementa un ciclo di controllo riconducibile al modello MAPE (*Monitoring, Analyzing, Planning, Execution*): monitora continuamente lo stato dei nodi, analizza le condizioni operative correnti, pianifica le decisioni di *placement* e le applica all'infrastruttura. Il parametro decisionale principale è il livello di carica dei nodi (*State of Charge*, SoC), che riflette la disponibilità energetica residua di ciascun elemento dell'infrastruttura.

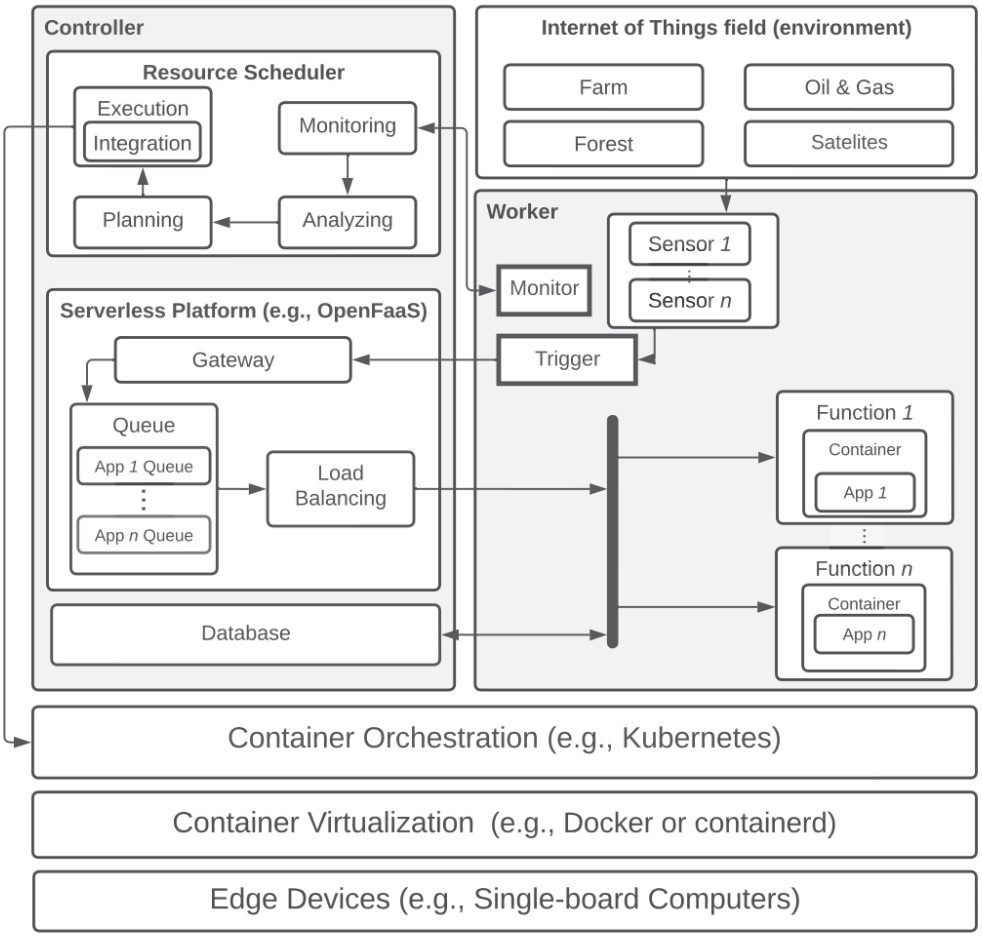

Figura 2.4: Architettura del sistema Faashouse. Il controller integra un *Resource Scheduler* che, seguendo un ciclo di monitoraggio, analisi e pianificazione, decide il *placement* delle funzioni sui nodi worker in base allo stato energetico corrente.

Dal punto di vista modellistico, lo stato energetico di ciascun nodo è una grandezza variabile nel tempo, determinata congiuntamente dall'energia dissipata durante l'esecuzione delle funzioni e dall'eventuale energia generata localmente. Le decisioni di assegnazione incidono direttamente sull'evoluzione di tale stato: concentrare le invocazioni su nodi con SoC ridotto può comprometterne la disponibilità futura, mentre una redistribuzione bilanciata consente di preservare l'operatività dell'intero sistema. Quando il livello di carica di un nodo si avvicina a una soglia critica, le nuove ri-

chieste vengono reindirizzate verso nodi più stabili mediante meccanismi di offloading, realizzando un bilanciamento energetico dinamico.

È opportuno sottolineare che Faashouse opera a granularità di nodo e non di funzione: le invocazioni sono trattate come unità indivisibili e non sono previste varianti della stessa funzione con profili energetici differenziati. L'ottimizzazione si realizza esclusivamente attraverso la selezione del nodo di esecuzione, senza intervenire sulla logica applicativa. Questa scelta progettuale delimita l'ambito di applicabilità del sistema: in scenari in cui il consumo energetico per invocazione dipende dalla complessità computazionale della funzione, la sola redistribuzione del carico potrebbe risultare insufficiente per soddisfare vincoli energetici stringenti a livello di singola esecuzione.

2.2.1.2 Scheduling per consolidamento energetico con vincoli di deadline

Accanto agli approcci orientati al bilanciamento della disponibilità energetica dei nodi, la letteratura propone strategie che perseguono l'efficienza energetica attraverso il *workload consolidation*. In questo modello, lo scheduler non mira a distribuire uniformemente il carico tra i nodi disponibili, bensì a concentrarlo su un sottoinsieme selezionato, riducendo il numero di elementi attivi e, di conseguenza, il consumo energetico complessivo dell'infrastruttura.

Un esempio rappresentativo di questo paradigma è E^2FIS (*Energy-Efficient Function Invocation Scheduling*) [19], proposto per piattaforme FaaS nell'edge–cloud continuum. Come illustrato in Figura 2.5, il sistema assume un insieme di nodi eterogenei, ciascuno caratterizzato da capacità computazionale, memoria disponibile, throughput effettivo e profilo di consumo energetico. Le funzioni sono descritte tramite requisiti di memoria, carico computazionale e deadline di completamento; la stima del tempo di esecuzione tiene conto delle differenze architetturali tra nodi, consentendo di verificare

la compatibilità di ciascuna assegnazione con i vincoli temporali imposti.

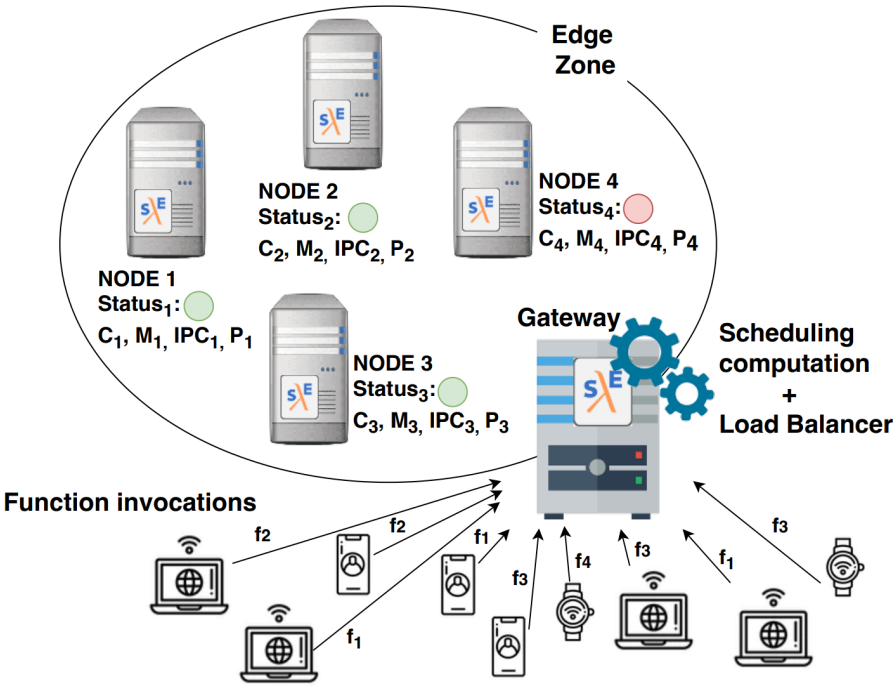

Figura 2.5: Architettura di E^{2}FIS. I nodi edge sono caratterizzati da differenti capacità computazionali e profili di consumo. Il modulo di *Scheduling computation* determina periodicamente l'allocazione delle funzioni e il sottoinsieme di nodi da mantenere attivi.

L'approccio adotta una pianificazione periodica basata su epoche temporali discrete. Prima dell'inizio di ciascuna epoca, il problema di allocazione viene formulato come un'istanza di Programmazione Lineare Intera Mista (MILP) e risolto offline. La funzione obiettivo minimizza il consumo energetico associato ai nodi attivi nell'intervallo considerato, selezionando il sottoinsieme minimo necessario a soddisfare tutte le richieste entro le rispettive deadline. Ne deriva una logica di consolidamento: quando il carico complessivo lo consente, le invocazioni vengono concentrate sui nodi con il miglior rapporto tra capacità computazionale e consumo energetico, mentre i restanti possono essere portati in stato inattivo.

Per garantire robustezza operativa in presenza di istanze computazionalmente one-

rose, il sistema prevede un limite temporale per la risoluzione del problema di ottimizzazione: qualora la soluzione ottima non sia ottenibile entro la soglia stabilita, viene adottata la migliore soluzione ammissibile disponibile. In scenari di carico elevato o con vincoli particolarmente stringenti, il framework può inoltre ricorrere a una politica di fallback ispirata a EDF (*Earliest Deadline First*), privilegiando il rispetto delle deadline rispetto all'ottimizzazione energetica. Tale meccanismo evidenzia come, in condizioni limite, il rispetto dei vincoli temporali costituisca un obiettivo prioritario rispetto all'efficienza energetica, introducendo una gerarchia esplicita tra gli obiettivi del sistema.

Nel complesso, E^2FIS rappresenta un approccio di scheduling energeticamente consapevole a granularità di nodo, nel quale il consumo energetico è trattato come metrica di ottimizzazione piuttosto che come vincolo fisico esplicito. L'adattamento si realizza esclusivamente attraverso la selezione del sottoinsieme di nodi attivi e l'assegnazione delle invocazioni, senza intervenire sulla logica applicativa né prevedere varianti delle funzioni con profili energetici differenziati. Analogamente a Faashouse, il perimetro decisionale rimane quindi circoscritto al livello infrastrutturale, lasciando inesplorata la possibilità di modulare il consumo energetico agendo direttamente sull'implementazione delle funzioni invocate.

2.2.1.3 Scheduling Energy-Aware in Ambienti FaaS Federati ed Eterogenei

In ambienti FaaS federati o multi-sito, la decisione di *placement* non dipende soltanto dall'efficienza del nodo di calcolo, ma anche dal costo energetico necessario a rendere disponibili i dati di input nel sito selezionato. In questi scenari, risorse eterogenee e geograficamente distribuite — tra cui cluster HPC, workstation e infrastrutture cloud — presentano profili energetici differenti, e le funzioni possono operare su dataset

residenti in domini amministrativi distinti. Ne consegue che l'allocazione ottimale deve considerare congiuntamente il costo di *computation* e di *communication*, evitando scelte che riducono il consumo di esecuzione introducendo al contempo overhead energetici significativi dovuti alla movimentazione dei dati.

In questo contesto si colloca GreenFaaS [20], un framework progettato per orchestrare invocazioni FaaS in ambienti distribuiti, integrando nella decisione di scheduling una stima esplicita del costo energetico dei trasferimenti. Come mostrato in Figura 2.6, l'architettura si articola in tre componenti principali: un modulo di *Resource Monitoring*, un *Transfers Manager* e uno *Scheduler*.

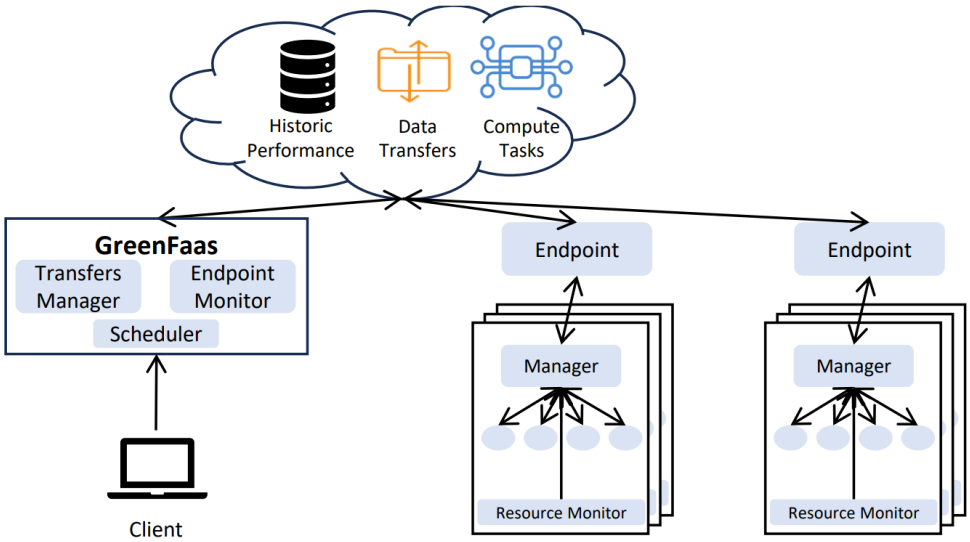

Figura 2.6: Architettura di alto livello di GreenFaaS: monitoraggio degli endpoint, stima del costo energetico dei trasferimenti e scheduling energy-aware per l'allocazione dei task in ambienti federati eterogenei.

Il modulo di monitoraggio raccoglie a runtime metriche energetiche e prestazionali dei nodi, sfruttando interfacce hardware — tra cui RAPL per CPU e NVML per GPU — per derivare stime di potenza e caratterizzare il comportamento energetico

delle risorse disponibili. Tali misure consentono di stimare il costo di esecuzione su piattaforme eterogenee, riducendo la dipendenza da proxy indiretti quali il solo tempo di esecuzione.

Il *Transfers Manager* modella il contributo energetico associato alla movimentazione dei dati tra siti. Quando l'input risiede in un dominio diverso da quello candidato all'esecuzione, il trasferimento introduce un costo legato alla trasmissione in rete e alle operazioni di I/O agli estremi della comunicazione. GreenFaaS stima tale costo in funzione della dimensione dei dati e delle caratteristiche del collegamento, consentendo di quantificare l'impatto energetico della scelta del sito anche nei casi in cui il nodo remoto risulti computazionalmente più efficiente.

Sulla base di queste informazioni, lo *Scheduler* effettua una selezione multi-criterio che integra il consumo energetico previsto per l'esecuzione sul nodo candidato e il costo energetico dei trasferimenti eventualmente necessari, affiancando a tali grandezze una stima del tempo di completamento. Il peso relativo delle componenti è configurabile, consentendo di modulare il compromesso tra efficienza energetica e prestazioni in funzione degli obiettivi applicativi. Per mantenere la reattività in presenza di carichi dinamici e infrastrutture di grandi dimensioni, il framework adotta strategie euristiche che evitano un'ottimizzazione globale onerosa. Nei contesti in cui il consumo in stato di inattività risulti significativo, sono inoltre previsti meccanismi di raggruppamento di task con profili simili, con l'obiettivo di ammortizzare il costo fisso di attivazione su esecuzioni consecutive.

Nel complesso, GreenFaaS amplia la nozione di consapevolezza energetica nello scheduling FaaS oltre la sola scelta del nodo computazionale, estendendola al costo di comunicazione tra siti. Questo contributo è particolarmente rilevante in architetture federali con forte dispersione geografica, dove i trasferimenti dati possono dominare il

bilancio energetico complessivo e rendere subottimali le scelte basate esclusivamente sull'efficienza di calcolo locale.

2.2.2 Power capping come tecnica di controllo energetico

Una delle tecniche più consolidate per la gestione del consumo energetico nei sistemi di calcolo distribuiti è il *power capping*, ovvero l'imposizione di un limite superiore alla potenza assorbita da uno o più componenti dell'infrastruttura. A differenza degli approcci orientati all'ottimizzazione dell'efficienza energetica in senso *best-effort*, il power capping introduce il consumo di potenza come vincolo operativo esplicito (*hard constraint*), che il sistema non deve violare durante l'esecuzione dei carichi di lavoro. L'adozione di tali meccanismi è motivata da vincoli infrastrutturali concreti: i limiti fisici dell'impianto elettrico, le capacità massime dei sistemi di raffreddamento o la necessità di operare con budget energetici limitati, come in scenari Edge o IoT. In letteratura, il problema è classicamente formulato come un'ottimizzazione vincolata, in cui l'obiettivo consiste nel massimizzare le prestazioni complessive del sistema nel rispetto di un budget di potenza prefissato $P_{\max}$.

2.2.2.1 Applicabilità degli approcci tradizionali nel contesto serverless

Approcci consolidati, come quelli analizzati da Ciesielczyk et al. [21], sfruttano leve hardware quali il *Dynamic Voltage and Frequency Scaling* (DVFS) o interfacce dedicate come Intel RAPL per modulare dinamicamente la frequenza dei processori e limitare il consumo di potenza. Tali tecniche si sono dimostrate efficaci in contesti HPC e in data center tradizionali, operando prevalentemente a livello di nodo fisico con una granularità temporale relativamente grossolana.

L'applicazione di questi meccanismi al paradigma serverless introduce tuttavia criticità specifiche. La natura effimera, *stateless* e altamente granulare delle funzioni FaaS

rende le politiche di capping statiche spesso poco reattive rispetto alla forte variabilità del carico di lavoro. Inoltre, il disaccoppiamento tra l'esecuzione delle funzioni e la gestione diretta delle risorse hardware limita la possibilità di applicare controlli di potenza con una precisione coerente con la scala temporale delle invocazioni serverless.

2.2.2.2 Impatto sulle prestazioni e approcci adattivi

L'imposizione di vincoli di potenza in ambienti serverless può comportare degradazioni delle prestazioni non lineari, riconducibili a due categorie di effetti ben documentate in letteratura [22].

La prima riguarda la latenza delle funzioni: esiste una correlazione diretta tra la riduzione della frequenza della CPU, indotta dal power capping, e l'aumento della latenza *end-to-end*. Tale effetto è particolarmente marcato per workload *CPU-bound*, mentre le funzioni *I/O-bound* risultano meno sensibili, poiché il tempo di esecuzione è dominato da operazioni di attesa piuttosto che di elaborazione.

La seconda criticità riguarda l'interazione con le politiche di autoscaling basate su metriche di utilizzo. In presenza di throttling energetico, il rallentamento dell'esecuzione viene spesso interpretato come un incremento del carico, inducendo decisioni di scaling subottimali. Questo comportamento è particolarmente problematico in sistemi adattivi, dove la non-stazionarietà introdotta dal vincolo di potenza può compromettere l'efficacia dei meccanismi di apprendimento e di controllo. La Tabella 2.2 sintetizza questi effetti, evidenziandone l'intensità differenziata in funzione del tipo di workload e della politica di gestione delle risorse considerata.

Per mitigare questi effetti, Qiu et al. propongono PARM (*Adaptive Resource Allocation for Datacenter Power Capping*) [22], un framework adattivo che interviene sulla catena decisionale dell'autoscaling in presenza di vincoli di potenza attivi. Come illu-

Tabella 2.2: Sintesi degli effetti del power capping sulle prestazioni in ambienti serverless (rielaborazione basata su [22]).

Ambito	**Effetto osservato**	**Intensità impatto**
Latenza CPU-bound	Aumento quasi lineare rispetto alla riduzione di frequenza.	Alto
Latenza I/O-bound	Impatto marginale, limitato alle fasi di elaborazione.	Basso
Autoscaling	Decisioni subottimali dovute a metriche non stazionarie.	Critico
SLO	Incremento delle violazioni degli accordi di servizio.	Critico

strato in Figura 2.7, PARM introduce un ciclo di controllo che opera in coordinamento con il meccanismo di power capping fisico, articolato in due componenti principali: un *Degradation Predictor*, che stima a runtime l'impatto del throttling sulle prestazioni applicative, e un *Resource Re-allocator*, che intercetta e corregge le decisioni dell'autoscaler prima che vengano applicate al livello di orchestrazione.

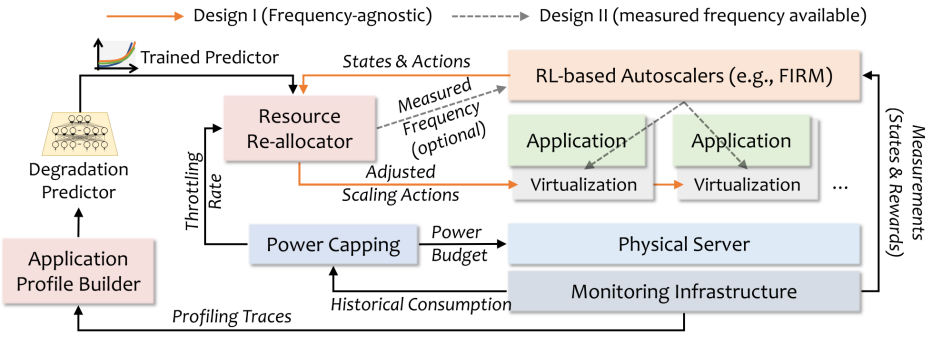

Figura 2.7: Schema architetturale del framework PARM. Il *Degradation Predictor* stima l'impatto del power capping sulle prestazioni, mentre il *Resource Re-allocator* adatta dinamicamente le decisioni di scaling in presenza di vincoli di potenza.

Nel complesso, il power capping rappresenta un meccanismo efficace per garantire il rispetto di vincoli energetici espliciti, ma la sua applicazione in ambienti serverless richiede componenti di controllo adattivo capaci di compensare le distorsioni che il

throttling introduce nei processi decisionali dell'infrastruttura. Gli approcci esaminati operano prevalentemente a livello hardware e di orchestrazione, agendo sulla potenza erogata ai nodi o sulle politiche di scaling, senza intervenire sulla logica di esecuzione delle singole funzioni.

2.2.3 Direzioni emergenti per l'elaborazione serverless sostenibile

Oltre agli interventi infrastrutturali descritti in precedenza, lo stato dell'arte evidenzia direzioni complementari volte a rendere l'esecuzione serverless più sostenibile attraverso meccanismi che agiscono a livello applicativo piuttosto che infrastrutturale. Anziché intervenire sulla distribuzione del carico o sulla potenza erogata ai nodi, tali approcci modificano la relazione tra il comportamento dell'applicazione e il consumo energetico, introducendo forme di controllo più fine.

Una prima direzione è rappresentata dal paradigma dell'*Approximate Computing*, che consente di ridurre il consumo energetico accettando, in modo controllato, una diminuzione dell'accuratezza del risultato. In questo ambito si colloca JouleGuard [7], che propone un modello di controllo runtime capace di rispettare un budget energetico prefissato regolando dinamicamente il livello di accuratezza dell'applicazione.

Come illustrato in Figura 2.8, il tratto distintivo di JouleGuard consiste nel trattare l'energia non come una metrica da minimizzare, ma come un vincolo esplicito che guida il comportamento del sistema a runtime. La selezione del livello di approssimazione non è determinata staticamente, ma dipende dalle condizioni operative osservate, rendendo il sistema adattivo rispetto alla disponibilità energetica corrente.

Una seconda direzione emerge dal lavoro *Code Once, Run Green* [23], che propone di intervenire sull'implementazione delle funzioni serverless mantenendone invariata la

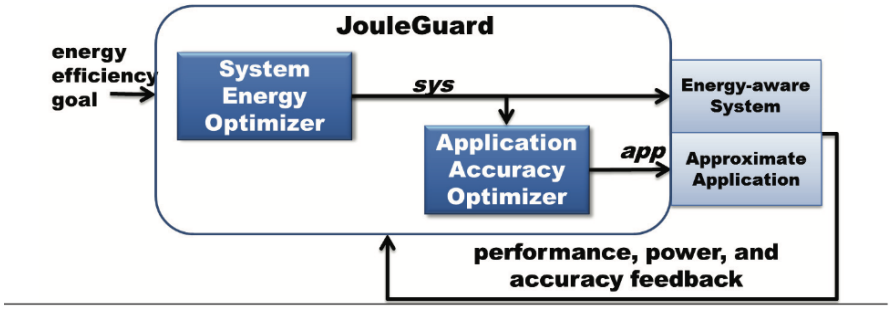

Figura 2.8: Schema concettuale di JouleGuard. Il sistema regola dinamicamente il livello di accuratezza dell'applicazione in funzione di un vincolo energetico esplicito, trattando il budget di potenza come parametro attivo nel processo di controllo.

semantica. Gli autori considerano la medesima funzione logica e ne realizzano una traduzione automatica verso un linguaggio di programmazione differente, consentendone l'esecuzione su runtime alternativi con potenziali profili energetici più favorevoli.

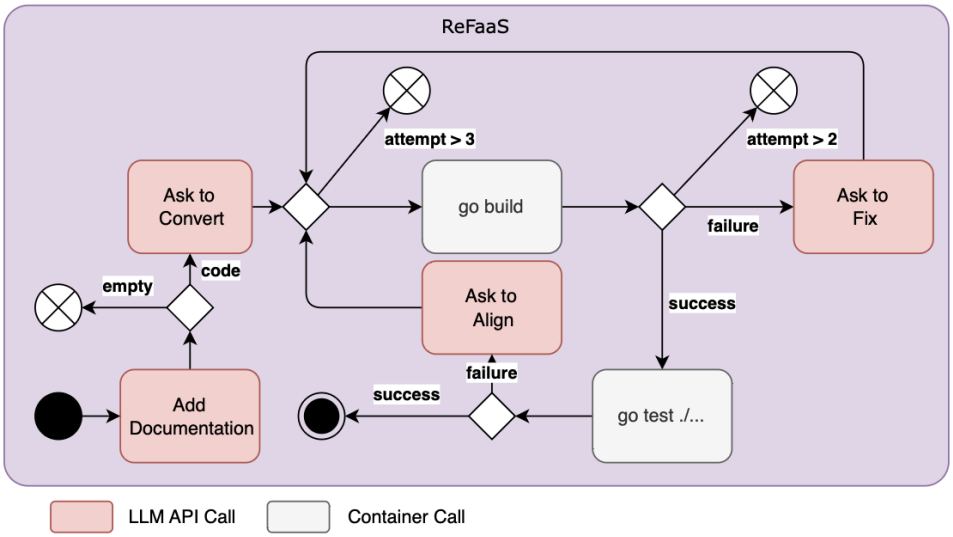

Figura 2.9: Pipeline di traduzione automatica proposta in *Code Once, Run Green*. La trasformazione avviene in fase di build tramite un processo iterativo di conversione e validazione del codice.

Come mostrato in Figura 2.9, la trasformazione del codice avviene in fase di build tramite una pipeline di conversione e validazione iterativa. A differenza dell'approc-

cio di JouleGuard, l'ottimizzazione energetica è qui perseguita staticamente, agendo sulla scelta del runtime piuttosto che sul comportamento dell'applicazione durante l'esecuzione. L'approccio introduce un costo energetico iniziale associato alla traduzione, potenzialmente ammortizzabile nel tempo attraverso la riduzione del consumo per invocazione della funzione tradotta.

Nel loro insieme, i lavori esaminati mostrano come la sostenibilità dell'elaborazione serverless possa essere perseguita anche al di fuori degli interventi infrastrutturali, attraverso meccanismi che agiscono sulla logica applicativa. JouleGuard introduce il concetto di vincolo energetico esplicito come parametro attivo nel controllo runtime; *Code Once, Run Green* mostra come la generazione di varianti della stessa funzione, eseguite su runtime differenti, possa produrre profili energetici significativamente diversi a parità di semantica.

2.3 Il paradigma FaaS in ambienti cloud-edge

Negli ambienti cloud-edge l'elaborazione è distribuita su un insieme eterogeneo di nodi di calcolo, caratterizzati da differenti capacità computazionali, disponibilità di memoria, ampiezza di banda e latenza di rete. Nel paradigma Function-as-a-Service, le applicazioni sono scomposite in funzioni stateless invocate su richiesta, la cui esecuzione è demandata alla piattaforma. In un contesto cloud-edge, ciascuna funzione può essere eseguita localmente su un nodo edge, oppure instradata verso altri nodi o verso il cloud, in funzione dello stato delle risorse disponibili al momento dell'invocazione.

Le differenze strutturali tra nodi cloud e nodi edge rendono il consumo energetico un fattore particolarmente rilevante in questi scenari. I nodi edge dispongono tipicamente di risorse limitate e operano in condizioni di maggiore variabilità del carico, spesso in assenza di infrastrutture di raffreddamento dedicate. Di conseguenza, le decisioni di

allocazione e gestione delle funzioni non possono basarsi esclusivamente su metriche prestazionali, ma devono tenere conto dell'impatto energetico delle scelte effettuate, soprattutto in presenza di vincoli operativi stringenti.

Le tecniche discusse nelle sezioni precedenti — scheduling energeticamente consapevole, power capping e meccanismi applicativi basati su approximate computing — sono state sviluppate prevalentemente in contesti cloud o HPC, con assunzioni sull'infrastruttura che non sempre si trasferiscono direttamente all'ambiente cloud-edge. In particolare, la modularità della piattaforma serverless di riferimento è un requisito determinante: affinché meccanismi di controllo energetico possano essere integrati senza alterare il modello di interazione lato utente, l'architettura della piattaforma deve esporre punti di estensione adeguati a livello di scheduling e gestione delle funzioni.

In questa prospettiva, la scelta della piattaforma serverless su cui condurre la sperimentazione non è indifferente. Nella sezione seguente viene presentata Serverledge, la piattaforma adottata come riferimento nel presente lavoro, descrivendone le caratteristiche architetturali rilevanti ai fini della ricerca.

2.3.1 La piattaforma Serverledge

Serverledge è una piattaforma Function-as-a-Service progettata per ambienti cloud-edge, caratterizzata da un'architettura decentralizzata che consente l'esecuzione di funzioni serverless all'interno di container. La piattaforma separa le componenti di controllo da quelle di esecuzione, permettendo una gestione distribuita delle istanze e delle informazioni di stato tra i nodi che compongono l'infrastruttura.

Ogni nodo Serverledge espone un'interfaccia per la creazione, l'eliminazione e l'invocazione delle funzioni, supportando sia la modalità sincrona sia quella asincrona. Le decisioni di esecuzione sono affidate a uno scheduler, responsabile del *placement* delle

richieste sui nodi disponibili. L'esecuzione avviene tramite container Docker, mantenuti attivi per un intervallo di tempo limitato successivamente all'invocazione, con l'obiettivo di ridurre l'impatto dei *cold start* sulle invocazioni successive.

La condivisione delle informazioni tra nodi è realizzata tramite un meccanismo di storage distribuito, che consente di propagare metadati e informazioni di stato all'interno del cluster. Tali informazioni sono accessibili dallo scheduler al momento della decisione di *placement*, permettendo di scegliere tra l'esecuzione locale sul nodo ricevente e l'offloading verso altri nodi edge o verso il cloud, in funzione delle condizioni operative rilevate.

Dal punto di vista architetturale, Serverledge è progettata per essere estensibile: le logiche di scheduling e di gestione delle funzioni sono separate dal modello di interazione lato utente, rendendo possibile l'integrazione di componenti aggiuntivi di monitoraggio e controllo senza modificare l'interfaccia esposta agli utenti. Questa proprietà è particolarmente rilevante nel contesto del presente lavoro, in quanto consente di introdurre meccanismi di gestione energetica direttamente a livello di scheduler, sfruttando i punti di estensione nativi della piattaforma senza alterarne il comportamento di base.

2.4 Posizionamento del contributo

L'analisi della letteratura evidenzia come le soluzioni proposte per la gestione energetica in ambienti serverless operino prevalentemente a livello infrastrutturale: gli scheduler energeticamente consapevoli esaminati — Faashouse, E^2FIS e GreenFaaS — agiscono sulla selezione del nodo di esecuzione e sulla redistribuzione del carico, trattando ciascuna funzione come un'unità indivisibile con un profilo energetico fisso. Analogamente, le tecniche di power capping intervengono sulla potenza erogata ai componenti hardware, senza alcuna consapevolezza della logica applicativa delle funzioni invocate. Contributi

come JouleGuard e *Code Once, Run Green* mostrano come sia possibile agire direttamente sulla funzione per ridurne il consumo, introducendo rispettivamente il controllo del livello di approssimazione e la generazione di implementazioni alternative. Tuttavia, tali approcci determinano i profili energetici delle varianti in modo prevalentemente statico, senza prevedere meccanismi di aggiornamento basati sul comportamento reale del sistema in esecuzione.

Questa limitazione è particolarmente critica in ambienti FaaS, nei quali il consumo energetico di una funzione non è una grandezza stabile nel tempo. Fattori quali la variabilità del carico concorrente sul nodo, le fluttuazioni termiche dell'hardware, il comportamento dei dati di input e lo stato degli ambienti di esecuzione possono determinare variazioni significative del consumo per invocazione anche a parità di implementazione. Affidarsi a stime statiche significa quindi operare su profili energetici potenzialmente disallineati rispetto alle condizioni operative correnti, compromettendo l'efficacia di qualsiasi meccanismo decisionale che si fonda su queste stime.

Il presente lavoro si propone di colmare questo spazio, introducendo un control loop runtime che mantiene i profili energetici delle varianti di funzione continuamente aggiornati sulla base di misurazioni reali, e li utilizza per prendere decisioni online di selezione. Tale approccio consente di combinare due dimensioni attualmente non integrate nella letteratura esaminata: la possibilità di sfruttare il trade-off tra consumo energetico e accuratezza offerto da varianti alternative della stessa funzione, e la capacità di rispettare vincoli energetici espliciti definiti dall'utente per singola invocazione. Quest'ultima caratteristica risulta particolarmente rilevante in scenari di esecuzione su nodi edge, dove la disponibilità energetica può variare nel tempo e la capacità di operare entro un budget predefinito rappresenta un requisito operativo concreto, non una semplice preferenza di ottimizzazione.

Capitolo 3

Progettazione e Realizzazione

In questo capitolo vengono descritte le scelte progettuali e i meccanismi realizzati per estendere Serverledge con capacità di monitoraggio e schedulazione energeticamente consapevole. Viene dapprima introdotto il modello di funzione logica con varianti energetiche, che costituisce l'astrazione fondamentale su cui si basa l'intero sistema. Si descrive quindi l'integrazione di Kepler per la misurazione dei consumi a livello di container e il worker periodico responsabile dell'aggiornamento adattivo dei profili energetici delle varianti. Infine, viene illustrata la logica decisionale dello scheduler energy-aware, progettato per selezionare a runtime la variante più accurata nel rispetto di un eventuale vincolo energetico, subordinatamente al consenso esplicito dell'utente.

3.1 Integrazione di Kepler per il monitoraggio energetico

Il monitoraggio dei consumi energetici a livello di container costituisce il fondamento del sistema realizzato. A tal fine è stato adottato Kepler (Kubernetes-based Efficient Power Level Exporter), uno strumento open source sviluppato nell'ambito della CNCF (Cloud Native Computing Foundation) che stima il consumo energetico di processi, container e pod Kubernetes attraverso l'accesso diretto alle interfacce hardware del

sistema operativo host [24].

La scelta di Kepler è motivata principalmente dalla sua granularità di misurazione, che opera a livello di singolo container Docker: questa corrisponde esattamente all'unità di esecuzione delle funzioni serverless in Serverledge. Kepler espone inoltre le proprie metriche nel formato standard Prometheus, consentendo un'integrazione diretta con l'ecosistema di monitoraggio senza richiedere componenti aggiuntivi per la raccolta dei dati.

3.1.1 Funzionamento di Kepler

Kepler opera combinando due meccanismi complementari per associare i consumi energetici ai singoli container. Il primo è un programma eBPF (Extended Berkeley Packet Filter) integrato nel kernel, che traccia l'attività dei processi e li associa ai container tramite il file `/proc/PID/cgroup`, consentendo di correlare il Process ID di ciascun processo all'identificatore del container che lo ospita. Il secondo è l'interfaccia RAPL (Running Average Power Limit), disponibile sui processori Intel e AMD moderni, che permette di leggere i contatori energetici cumulativi direttamente dai registri hardware della CPU.

Una volta raccolti i dati di utilization dei processi e di consumo energetico del sistema, Kepler applica il *Ratio Power Model* per attribuire a ciascun container la propria quota di consumo: la potenza dinamica viene distribuita tra i container in proporzione alla loro utilization rispetto all'utilization totale del sistema. Il risultato viene aggregato a livello di container e reso disponibile tramite Prometheus nella metrica `kepler_container_cpu_joules_total`, un contatore cumulativo espresso in Joule che cresce monotonicamente nel tempo per ogni container attivo.

Come illustrato in Figura 3.1, il comportamento di Kepler varia in base all'ambiente

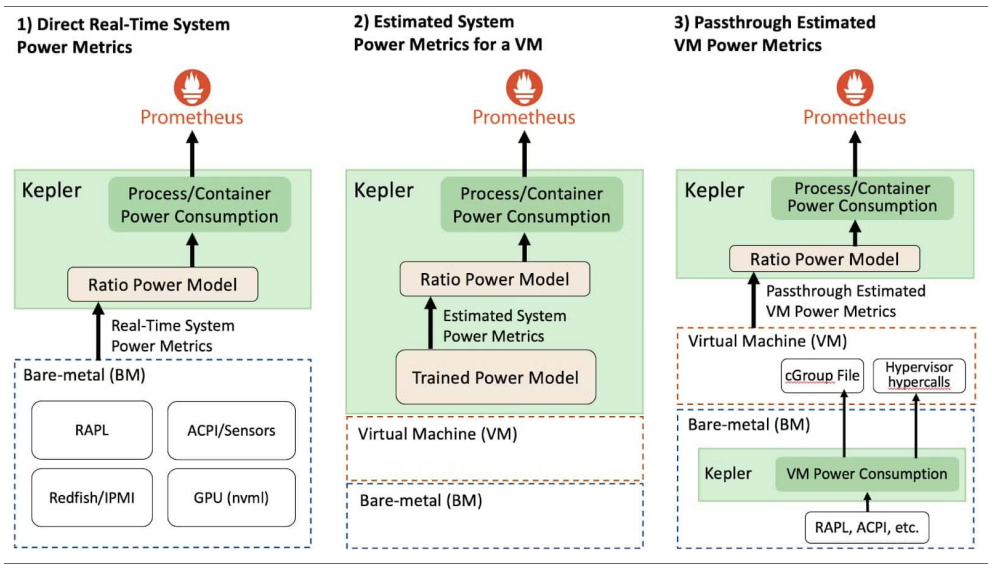

Figura 3.1: Architettura di Kepler per la raccolta dei consumi energetici in ambienti bare-metal e virtuali. In ambienti bare-metal, Kepler accede direttamente ai contatori hardware tramite RAPL, ottenendo misurazioni real-time senza necessità di modelli predittivi.

di esecuzione. In ambienti bare-metal, come quello utilizzato in questo lavoro, Kepler accede direttamente ai contatori RAPL, ottenendo misurazioni energetiche real-time senza ricorrere a modelli predittivi pre-addestrati. Questo garantisce una stima più accurata rispetto agli ambienti virtualizzati, dove l'assenza di accesso diretto all'hardware richiede l'uso di modelli di regressione con le relative limitazioni di accuratezza.

Per accedere alle interfacce hardware necessarie, Kepler deve essere eseguito con privilegi elevati sull'host: richiede accesso a `/sys` e `/proc` per leggere i contatori RAPL e i cgroup, e al socket Docker per identificare i container attivi e associarvi le metriche energetiche.

3.1.2 Infrastruttura di supporto

L'avvio dell'infrastruttura di monitoraggio è gestito dallo script `StartMetrics.sh`, che inizializza come container Docker tutti i servizi necessari al funzionamento del sistema.

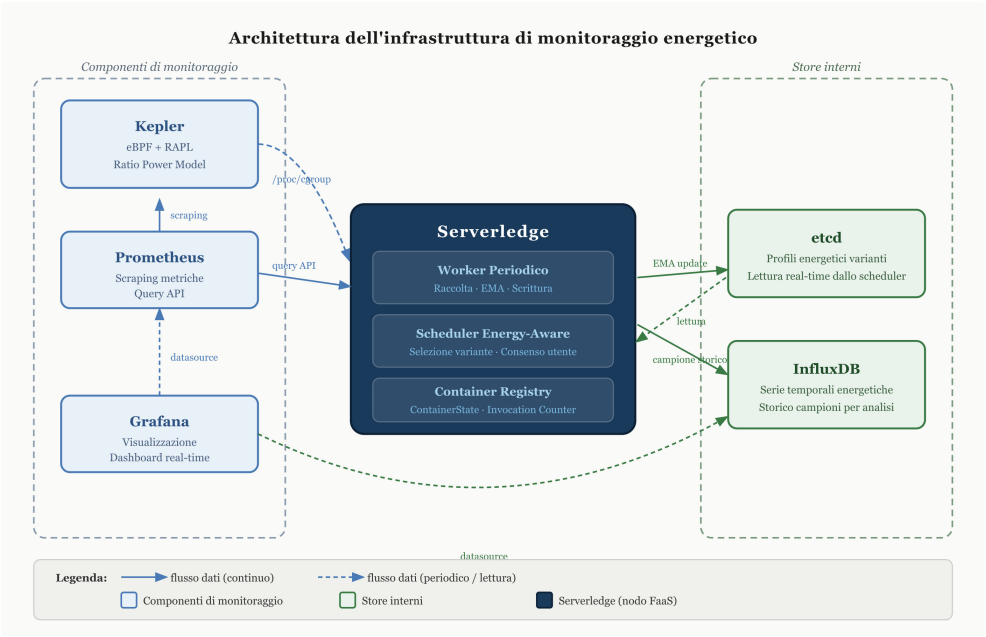

Figura 3.2: Architettura dell'infrastruttura di monitoraggio energetico. I componenti di monitoraggio esterni (Kepler, Prometheus, Grafana) sono separati dagli store interni (etcd, InfluxDB), con Serverledge al centro che interagisce con entrambi i livelli.

I componenti avviati, come mostrato anche in Figura 3.2, sono i seguenti:

- **Kepler**: eseguito in modalità privilegiata con accesso a `/sys`, `/proc` e al socket Docker dell'host, condizioni necessarie per la lettura dei contatori RAPL e l'associazione dei consumi ai container tramite cgroup.
- **Prometheus**: configurato per effettuare lo scraping periodico delle metriche esposte da Kepler, rendendole disponibili tramite API di interrogazione.
- **InfluxDB**: database a serie temporali utilizzato per l'archiviazione storica dei campioni energetici prodotti dal sistema ad ogni ciclo di raccolta.
- **etcd**: store distribuito chiave-valore che funge da registro delle funzioni e delle relative varianti, inclusi i profili energetici aggiornati dinamicamente a runtime.

- **Grafana**: strumento di visualizzazione utilizzato per il monitoraggio in tempo reale dei consumi durante gli esperimenti.

Una volta avviata, l'infrastruttura opera in modo autonomo: Kepler rileva i container Docker presenti sull'host e ne aggiorna le metriche energetiche ad ogni ciclo di campionamento; Prometheus le raccoglie e le rende interrogabili; il sistema interno di Serverledge, descritto nelle sezioni successive, le legge periodicamente per aggiornare i profili energetici delle varianti di funzione.

3.2 Il modello di funzione logica con varianti

Il supporto all'esecuzione energeticamente consapevole richiede che il sistema possa rappresentare, per una stessa funzione, implementazioni alternative con diversi profili di consumo e accuratezza. A tal fine è stato introdotto il concetto di *funzione logica con varianti*: un'astrazione che raggruppa sotto un unico identificatore logico più implementazioni della stessa funzione, ciascuna caratterizzata da un proprio profilo energetico e da un modello di output che ne quantifica l'accuratezza.

3.2.1 Struttura della funzione e profilo energetico

In Serverledge, ogni funzione è rappresentata dalla struttura `Function`, estesa in questo lavoro con i campi necessari al supporto delle varianti energetiche. I campi rilevanti per il sistema proposto sono i seguenti:

- `LogicalName`: identificatore condiviso da tutte le varianti appartenenti alla stessa funzione logica. Costituisce il riferimento con cui il client invoca la funzione, indipendentemente dalla variante che verrà selezionata a runtime.

- `VariantID`: identificatore univoco della specifica variante all'interno del gruppo logico (ad esempio `base`, `light`, `c`).
- `EnergyProfile`: struttura che mantiene la stima energetica della variante, composta da due campi:
  - `ColdStartJoule`: energia stimata per l'avvio a freddo del container, comprensiva dell'inizializzazione del runtime;
  - `InvocationJoule`: energia media stimata per una singola invocazione della funzione
- `AllowApprox`: flag booleano che indica se la funzione logica ammette l'utilizzo di varianti approssimate. La sua attivazione è condizione necessaria affinché lo scheduler energy-aware possa considerare varianti con accuratezza ridotta.

3.2.1.1 Determinazione dei profili energetici iniziali

I valori iniziali di `EnergyProfile` sono stati determinati offline tramite una campagna di misurazioni preliminari condotta con Kepler. La metodologia adottata tiene conto della natura delle metriche esposte da Kepler: trattandosi di contatori cumulativi, l'energia osservata al termine di n invocazioni consecutive sullo stesso container rappresenta la somma dei contributi di tutte le esecuzioni, inclusa quella iniziale soggetta al cold start.

Per separare i due contributi energetici si è proceduto nel modo seguente. Dato un container su cui vengono eseguite n invocazioni consecutive, si indica con E_{tot} l'energia cumulativa totale rilevata da Kepler al termine dell'ultima invocazione e con E_1 l'energia della prima invocazione, che comprende sia il costo di inizializzazione del container che quello di esecuzione della funzione. Le invocazioni successive, eseguite a

container già attivo, sono invece soggette al solo costo di esecuzione. Il costo medio di invocazione in warm start è quindi stimato come:

$$E_{inv} = \frac{E_{tot} - E_1}{n - 1} \tag{3.2.1}$$

e il costo di cold start come:

$$E_{cold} = E_1 - E_{inv} \tag{3.2.2}$$

La ripetizione delle misurazioni su un numero statisticamente significativo di invocazioni consente di attenuare la varianza introdotta dal sistema operativo e dal runtime, ottenendo stime robuste per entrambi i contributi. I valori così ottenuti vengono codificati nel file JSON di definizione delle varianti e costituiscono la stima iniziale del profilo energetico. Il Codice 3.1 mostra un esempio concreto per la funzione `fibonacci`, che espone tre varianti con profili energetici distinti.

Codice 3.1: File di definizione delle varianti per la funzione `fibonacci`. Ciascuna variante specifica runtime, entry point, profilo energetico iniziale e modello di output.

```
{
  "variants": [
    {
      "id": "base",
      "runtime": "python310",
      "entry_point": "fibonacci.handler",
      "src": "fibonacci.py",
      "energy": {
        "cold_start_joule": 0.06606,
        "invocation_joule": 0.000731579
      },
      "output": { "type": "error", "error_estimate": 0.0 },
      "is_approximate": false
    },
    {
      "id": "optimization-py",
      "runtime": "python310",
      "entry_point": "fibonacci_o.handler",
```

```
19        "src": "fibonacci_o.py",
20        "energy": {
21          "cold_start_joule": 0.05640,
22          "invocation_joule": 0.000494737
23        },
24        "output": { "type": "error", "error_estimate": 0.0 },
25        "is_approximate": false
26      },
27      {
28        "id": "c",
29        "runtime": "native",
30        "entry_point": "fibonacci",
31        "src": "fibonacci",
32        "energy": {
33          "cold_start_joule": 0.008268422,
34          "invocation_joule": 0.000171579
35        },
36        "output": { "type": "error", "error_estimate": 0.0 },
37        "is_approximate": false
38      }
39    ]
40  }
```

I valori iniziali del profilo energetico sono progressivamente raffinati a runtime dal worker periodico, che aggiorna entrambi i valori sulla base delle misurazioni effettive raccolte durante l'esecuzione in produzione. I risultati numerici delle misurazioni preliminari per tutte le varianti sono riportati nel Capitolo 4.

3.2.2 Il modello di output e la classificazione dell'accuratezza

Per consentire allo scheduler di confrontare le varianti in termini di accuratezza, ciascuna variante è dotata di un `OutputModel` che descrive la qualità del suo output. Il modello supporta due tipologie di classificazione, scelte in base alla natura computazionale della funzione:

- **Tipo** `error`: adatto a funzioni il cui output è un valore numerico misurabile. L'accuratezza è espressa tramite il campo `error_estimate`, che rappresenta

l'errore relativo atteso rispetto alla variante di riferimento. Una variante esatta presenta `error_estimate = 0.0`, mentre varianti approssimate presentano valori positivi proporzionali alla deviazione introdotta dalla semplificazione algoritmica adottata.

- **Tipo** `quality`: adatto a funzioni il cui output non è facilmente quantificabile con un errore numerico, come classificatori o modelli di machine learning. L'accuratezza è espressa tramite un livello discreto: `high`, `medium` o `low`, che riflette la qualità complessiva del risultato prodotto dalla variante.

Il Codice 3.2 mostra un esempio applicativo del tipo `quality` per la funzione `ImageClassification`, in cui la variante base utilizza un modello di rete neurale più accurato ma più oneroso, mentre la variante `Light` adotta un modello più compatto a qualità ridotta.

Codice 3.2: File di definizione delle varianti per la funzione `ImageClassification`. Il campo `quality` esprime l'accuratezza in forma discreta, adatta a output non numericamente quantificabili.

```
{
  "variants": [
    {
      "id": "base",
      "runtime": "python-ml",
      "entry_point": "ImageClassification.handler",
      "src": "ImageClassification.py",
      "energy": {
        "cold_start_joule": 1.894736842,
        "invocation_joule": 1.455263158
      },
      "output": { "type": "quality", "quality": "high" },
      "is_approximate": false
    },
    {
      "id": "Light",
      "runtime": "python-ml",
      "entry_point": "ImageClassificationLight.handler",
      "src": "ImageClassificationLight.py",
```

```
20        "energy": {
21          "cold_start_joule": 1.071052329,
22          "invocation_joule": 0.22894768
23        },
24        "output": { "type": "quality", "quality": "low" },
25        "is_approximate": true
26      }
27    ]
28  }
```

Lo scheduler converte internamente entrambe le rappresentazioni in un punteggio numerico (*ErrorScore*) secondo una scala uniforme: per il tipo `error` viene utilizzato direttamente il valore di `error_estimate`, mentre per il tipo `quality` i livelli `high`, `medium` e `low` vengono mappati rispettivamente a 0.0, 1.0 e 2.0. La selezione privilegia sempre la variante con il punteggio più basso compatibile con il vincolo energetico, garantendo che la qualità del risultato sia massimizzata nei limiti imposti dalla richiesta.

La Tabella 3.1 riassume il modello di output adottato per ciascuna delle funzioni utilizzate negli esperimenti.

Funzione	**Tipo**	**Variante base**	**Variante approssimata**
Fibonacci	`error`	0.0	0.0
PiLeibniz	`error`	0.0	1.99×10^{-8}
Sqrt	`error`	0.0	0.043
VectorMagnitude	`error`	0.0	0.04
SentimentAnalysis	`quality`	high	low
ImageClassification	`quality`	high	low

Tabella 3.1: Modello di output adottato per ciascuna funzione. Per il tipo `error` il valore riportato è `error_estimate`; per il tipo `quality` il livello discreto assegnato.

3.2.3 Registrazione e indicizzazione delle varianti in etcd

Le varianti di una funzione logica vengono definite tramite un file JSON collocato nella directory `variants/<LogicalName>/` del progetto. Al momento della registrazione,

il sistema legge questo file attraverso il componente `FileSource`, prepara il codice sorgente di ciascuna variante in formato tar e materializza le varianti in etcd attraverso la funzione `CreateInternalVariants`.

3.2.3.1 Convenzione di denominazione

La convenzione di denominazione distingue la variante di riferimento dalle varianti alternative. La prima variante definita nel file JSON — tipicamente quella a massima accuratezza — mantiene come nome interno esattamente il `LogicalName`, preservando la compatibilità con le invocazioni che non specificano alcuna preferenza energetica. Le varianti successive ricevono un nome composto nella forma `<LogicalName>-<VariantID>`. Per la funzione logica `fibonacci` (Codice 3.1) ad esempio, i nomi interni assegnati sono:

- `fibonacci` — variante base in Python
- `fibonacci-optimization-py` — variante ottimizzata in Python
- `fibonacci-c` — variante compilata in C nativo

3.2.3.2 Struttura delle chiavi in etcd

Il sistema mantiene in etcd due livelli distinti di chiavi, ciascuno con una responsabilità specifica, come illustrato in Figura 3.3.

Il primo livello, sotto il prefisso `/function/`, contiene una chiave per ogni variante registrata nel sistema, indipendentemente dalla funzione logica di appartenenza. Il valore associato è la serializzazione JSON completa della struttura `Function`, che include il runtime, il codice sorgente, il profilo energetico corrente e il modello di output. È su queste chiavi che il worker periodico scrive gli aggiornamenti del campo `InvocationJoule` ad ogni ciclo di raccolta.

Il secondo livello, sotto il prefisso `/index/logical/<LogicalName>/`, costituisce un indice che associa ciascun nome logico all'insieme delle sue varianti. La motivazione di questo indice è di natura prestazionale: poiché la selezione della variante avviene nel critical path della richiesta di invocazione, è necessario che lo scheduler possa recuperare rapidamente tutte le varianti di una funzione logica senza dover scorrere l'intero prefisso `/function/`, che contiene tutte le funzioni registrate nel sistema. Tramite l'indice, il recupero avviene con una singola query con prefisso su `/index/logical/<LogicalName>/`, restituendo esclusivamente le varianti della funzione richiesta e contenendo così la latenza introdotta dalla fase di selezione a runtime.

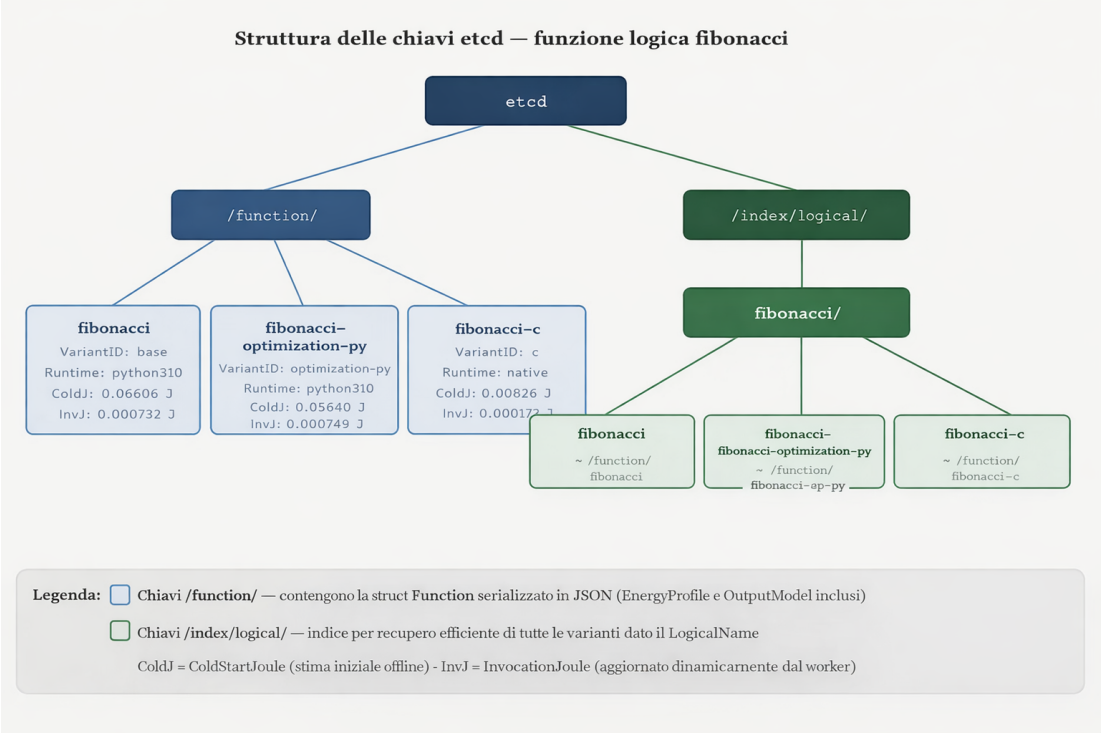

Figura 3.3: Struttura delle chiavi etcd per la funzione logica `fibonacci` con tre varianti. Le chiavi sotto `/function/` contengono i profili completi aggiornati dal worker periodico. L'indice sotto `/index/logical/` consente allo scheduler di recuperare efficientemente le sole varianti della funzione richiesta, senza scorrere l'intero store.

3.3 Misurazione e aggiornamento adattivo dei profili energetici

Una volta introdotto il modello di funzione logica con varianti multiple, è necessario dotare il sistema di un meccanismo capace di aggiornare dinamicamente i profili energetici sulla base dei consumi effettivamente osservati a runtime. Un approccio statico, in cui il costo energetico di una variante viene stimato offline e rimane invariato nel tempo, risulta inadeguato in contesti di esecuzione reali: il consumo energetico di un container è influenzato da fattori mutevoli quali il carico del nodo, le interferenze tra istanze concorrenti e le variazioni dinamiche della frequenza di CPU.

La soluzione proposta adotta invece un *feedback loop* periodico che trasforma il profilo energetico in una grandezza adattiva, aggiornata continuamente in funzione del contesto di esecuzione. Il componente responsabile di tale meccanismo è un worker di raccolta eseguito in background, che costituisce il nucleo del controllo retroazionato del sistema. La Figura 3.4 illustra il flusso completo di acquisizione, elaborazione e persistenza dei campioni energetici.

3.3.1 Tracciamento dei container e conteggio delle invocazioni

Per attribuire correttamente il consumo energetico alle singole varianti, il sistema deve mantenere una corrispondenza stabile tra i container Docker attivi e le funzioni in esecuzione. A tal fine è stato introdotto un registro in memoria, denominato `ContainerRegistry`, che associa a ciascun container una struttura `ContainerState` contenente i seguenti campi:

- l'identificatore del container (`ContainerID`);

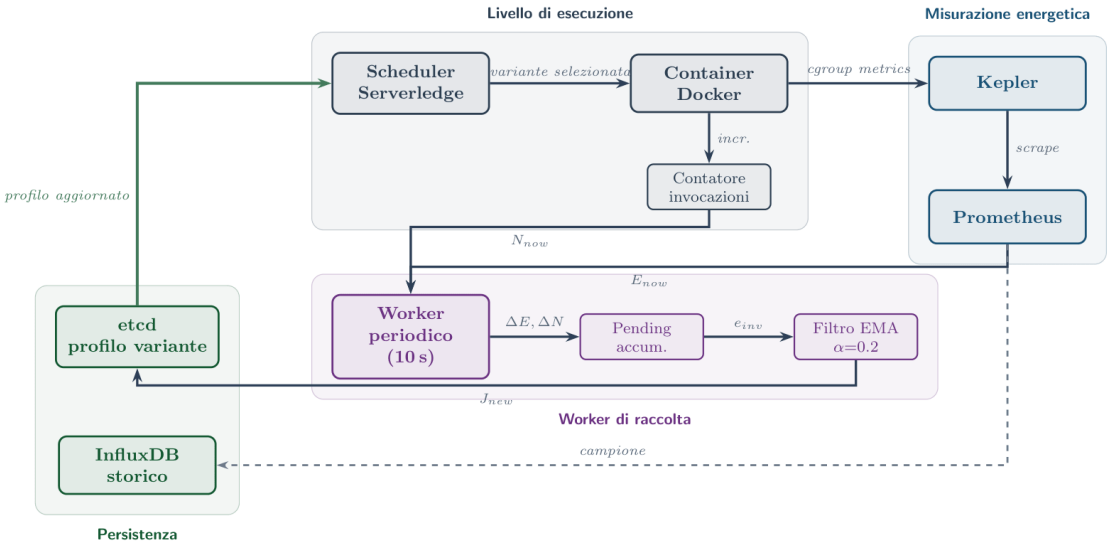

Figura 3.4: Schema del feedback loop energetico. Il worker periodico legge le metriche cumulative da Kepler tramite Prometheus e il numero di invocazioni dal livello di esecuzione, calcola i delta, applica un filtro EMA e aggiorna il profilo persistito in etcd, archiviando contestualmente i campioni in InfluxDB.

- il nome della variante (`FunctionName`), il relativo `LogicalName` e il `VariantID` che la identifica univocamente nella funzione logica;
- i valori di baseline dell'energia cumulativa e del numero di invocazioni, impiegati per il calcolo dei delta incrementali;
- gli accumulatori `PendingEnergyJ` e `PendingInvocations`, necessari per gestire il disallineamento temporale tra metriche energetiche e contatori locali.

La registrazione di un container avviene sia in fase di creazione del runtime sia, in modo difensivo, al momento dell'invocazione, così da gestire correttamente il caso di *warm start*, in cui il container è già attivo ma non ancora registrato. Il processo è tale per cui, se un container è già presente nel registry, il mapping container-variante non viene modificato, garantendo coerenza durante l'intero ciclo di vita del container.

Parallelamente, viene mantenuto un contatore atomico di invocazioni per container, incrementato al completamento di ogni richiesta. L'impiego di operazioni atomiche evita l'introduzione di meccanismi di lock nel *critical path* dello scheduler, preservando le prestazioni anche in presenza di elevata concorrenza.

3.3.2 Il worker periodico di raccolta

Il worker è implementato come una goroutine avviata all'inizializzazione della piattaforma. Con cadenza di 10 secondi, esegue un ciclo di raccolta (`Collect`) operando su uno snapshot thread-safe del `ContainerRegistry`, in modo da isolare la lettura dallo stato condiviso senza bloccare le operazioni concorrenti.

Per ciascun container attivo vengono acquisiti:

- il valore cumulativo di energia E_{now}, esposto da Kepler e reso disponibile tramite l'endpoint Prometheus;
- il numero cumulativo di invocazioni N_{now}, mantenuto aggiornato dal contatore locale.

Vengono quindi calcolati i delta rispetto all'osservazione precedente:

$$\Delta E = E_{now} - E_{last} \qquad \Delta N = N_{now} - N_{last} \tag{3.3.1}$$

La presenza di valori negativi segnala un reset del container o del contatore — condizione possibile ad esempio in seguito a un riavvio dell'istanza — e comporta la reinizializzazione della baseline senza produzione di campioni, evitando attribuzioni energetiche errate.

3.3.3 Attribuzione dell'energia per invocazione e gestione del lag

Le metriche esposte da Kepler hanno natura cumulativa e sono soggette a un ritardo di campionamento intrinseco al ciclo di scraping di Prometheus. Il contatore locale di invocazioni, al contrario, viene aggiornato immediatamente al termine di ogni esecuzione. Può quindi verificarsi la condizione:

$$\Delta N > 0 \quad \text{e} \quad \Delta E = 0$$

in cui nuove invocazioni sono già state completate ma il relativo consumo energetico non è ancora riflesso nella metrica cumulativa. Calcolare in questo caso il consumo medio per invocazione produrrebbe una sottostima sistematica.

Per gestire questo disallineamento, i delta vengono accumulati nei campi `PendingEnergyJ` e `PendingInvocations`. L'attribuzione effettiva avviene solo quando entrambi gli accumulatori risultano positivi, garantendo che l'energia osservata e il numero di esecuzioni considerate siano coerenti tra loro:

$$e_{inv} = \frac{E_{pending}}{N_{pending}} \tag{3.3.2}$$

Il valore e_{inv} rappresenta la stima istantanea del consumo medio per invocazione nel periodo considerato e costituisce l'ingresso per la fase di aggiornamento adattivo del profilo.

3.3.4 Aggiornamento del profilo tramite media mobile esponenziale

La stima istantanea e_{inv} può essere affetta da rumore dovuto a interferenze tra container, variazioni dinamiche della frequenza di CPU o fluttuazioni del carico di sistema.

Applicare direttamente tale valore al profilo energetico potrebbe causare oscillazioni indesiderate nelle decisioni dello scheduler.

Per stabilizzare la stima viene applicata una *Exponential Moving Average* (EMA):

$$J_{new} = \alpha \cdot e_{inv} + (1 - \alpha) \cdot J_{old} \tag{3.3.3}$$

con $\alpha = 0.2$. Il parametro α governa il compromesso tra due esigenze contrastanti:

- **Reattività**: valori elevati rendono il profilo rapidamente adattabile a variazioni strutturali del consumo energetico;
- **Stabilità**: valori ridotti attenuano il contributo di singoli campioni anomali, producendo una stima più robusta.

La scelta di $\alpha = 0.2$ rappresenta un equilibrio empirico: il sistema filtra efficacemente i picchi transitori, mantenendo al contempo una capacità di adattamento progressivo al nuovo regime energetico nell'arco di circa $1/\alpha$ cicli di aggiornamento, corrispondenti a circa 50 secondi con la cadenza attuale del worker.

Il valore aggiornato J_{new} viene scritto nel campo `InvocationJoule` della variante corrispondente e persistito in etcd, rendendolo immediatamente disponibile allo scheduler per le decisioni di selezione successive.

3.3.5 Archiviazione su InfluxDB

Contestualmente all'aggiornamento del profilo in etcd, ogni campione viene archiviato in InfluxDB come punto della *measurement* `energy_sample`. Ogni punto include tag identificativi del container e della variante, nonché il campo `invocation_joule` espresso in Joule.

La separazione tra etcd e InfluxDB riflette la doppia funzione che il dato energetico riveste nel sistema:

- etcd supporta il *presente*: fornisce allo scheduler la stima corrente con latenza minima, ottimizzando il percorso decisionale in fase di selezione della variante;
- InfluxDB preserva il *passato*: mantiene la serie storica dei campioni energetici, abilitando analisi sperimentali e visualizzazioni grafiche.

Questa architettura a doppio livello consente di disaccoppiare le esigenze operative dello scheduler — latenza e consistenza a runtime — dalle esigenze analitiche della fase sperimentale.

3.4 Lo scheduler energy-aware

Lo scheduler energy-aware costituisce il componente decisionale centrale dell'estensione progettata. Il suo compito è selezionare, tra le varianti registrate per una data funzione logica, quella da eseguire concretamente in risposta a ciascuna richiesta di invocazione. La selezione avviene prima dell'assegnazione del container e dell'avvio dell'esecuzione vera e propria, consentendo di sfruttare le stime energetiche aggiornate presenti in *etcd* come base per le proprie decisioni.

A differenza degli scheduler tradizionali, che operano esclusivamente a livello di selezione del nodo di esecuzione, lo scheduler progettato interviene a granularità di variante: a parità di funzione logica richiesta, è possibile eseguire implementazioni alternative caratterizzate da profili energetici e livelli di errore distinti. Questa scelta progettuale consente di trattare il trade-off tra consumo energetico e qualità del risultato come una grandezza controllabile a runtime, anziché come una proprietà fissa dell'implementazione.

Il comportamento dello scheduler è ispirato ai meccanismi di controllo runtime introdotti da JouleGuard [7], adattati al paradigma FaaS e al modello a varianti descritto nella Sezione 3.2. In particolare, il sistema non applica alcuna selezione alternativa in assenza di un'esplicita autorizzazione da parte dell'utente, garantendo la piena compatibilità con il comportamento legacy della piattaforma.

La logica decisionale dello scheduler si articola in tre casi distinti, determinati dai parametri forniti dall'utente al momento dell'invocazione: il caso base (senza abilitazione della variant selection), il caso di selezione per errore minimo e il caso di selezione con vincolo energetico esplicito. Tali casi sono analizzati nelle sezioni seguenti.

3.4.1 Il consenso esplicito dell'utente

Un principio fondamentale che guida la progettazione dello scheduler è la separazione netta tra il comportamento di default della piattaforma e la modalità energy-aware. La selezione di varianti alternative non viene attivata di default dalla piattaforma: essa richiede sempre un consenso esplicito da parte dell'utente che formula la richiesta di invocazione.

A tale scopo, l'interfaccia di invocazione è stata estesa con due campi aggiuntivi: il flag booleano `AllowApprox` e il parametro opzionale `MaxEnergyJoule`. Il primo esprime il consenso dell'utente all'utilizzo di varianti alternative rispetto a quella nominale; il secondo, se presente e positivo, specifica un budget energetico massimo accettabile per l'invocazione, espresso in Joule.

In assenza di autorizzazione esplicita — ovvero quando `AllowApprox = false` — la variant selection è completamente disabilitata e la richiesta viene instradata verso la variante specificata nella chiamata, secondo il comportamento originale della piattaforma. Questo garantisce la piena compatibilità retroattiva con le applicazioni preesistenti e

con gli utenti che non desiderano delegare allo scheduler la scelta dell'implementazione.

Quando invece `AllowApprox = true`, lo scheduler attiva la variant selection energy-aware. In questo caso, tutte le varianti registrate per la funzione logica corrispondente diventano candidate all'esecuzione e la selezione avviene secondo criteri energetici e di errore, eventualmente vincolati dal parametro `MaxEnergyJoule`. Il flusso di controllo ad alto livello è formalizzato nell'Algoritmo 1.

Algoritmo 1 `SubmitRequest` con variant selection opt-in

```
1: function SubmitRequest(r : *function.Request)
2:     if r.AllowApprox = true then
3:         (selectedFn, err) ← SelectEnergyAwareVariant(r)
4:         if err ≠ nil then
5:             return (nil, err)      ▷ vincolo energetico non soddisfatto → HTTP 422
6:         if selectedFn ≠ nil then
7:             r.Fun ← selectedFn                    ▷ sostituisce la variante richiesta
8:     requests ← r                          ▷ invia al loop legacy: p.OnArrival(r)
9:     return SchedulingDecision(r)
```

La struttura dell'algoritmo evidenzia come la variant selection si inserisca come un livello di pre-elaborazione rispetto allo scheduler originale: il risultato della selezione è la sostituzione della variante richiesta con quella ottimale individuata dalla funzione SelectEnergyAwareVariant, dopodiché l'esecuzione prosegue secondo il percorso ordinario. In caso di errore — tipicamente quando nessuna variante soddisfa i vincoli imposti — la richiesta viene rifiutata con un codice di stato HTTP 422 (*Unprocessable Entity*) senza avviare alcuna computazione.

3.4.2 Selezione per errore minimo

Quando l'utente abilita la variant selection (`AllowApprox = true`) senza specificare un budget energetico, oppure con un valore `MaxEnergyJoule` nullo o non positivo, lo scheduler opera secondo la politica di *accuracy-first*. In questo scenario, l'obiettivo

è individuare la variante che offre il risultato di qualità più elevata tra tutte quelle disponibili, ricorrendo al consumo energetico stimato esclusivamente nel caso di parità di errore minimo tra due o più varianti.

La politica non applica alcun filtraggio energetico preventivo: tutte le varianti registrate per la funzione logica sono candidate alla selezione, indipendentemente dal loro profilo energetico. Questo approccio è coerente con la natura opt-in del meccanismo: l'utente ha autorizzato l'uso di varianti alternative, ma non ha imposto alcun vincolo sul consumo; la piattaforma interpreta quindi tale consenso come un interesse prioritario verso la qualità del risultato.

Per confrontare varianti con modelli di output eterogenei viene definita una funzione di punteggio di errore $Err(v)$, costruita in modo tale che valori numericamente inferiori corrispondano a errore minore. La funzione è implementata nel componente `ErrorScore`(v) secondo le regole seguenti:

- se il modello di output della variante è di tipo `"error"`, il punteggio è pari alla stima dell'errore associato: $Err(v) = ErrorEstimate(v)$;
- se il modello è di tipo `"quality"`, la classe qualitativa viene convertita in una scala numerica ordinale: $High \to 0$, $Medium \to 1$, $Low \to 2$;
- in assenza di un modello di output esplicito, si pone per convenzione $Err(v) = 0$, attribuendo l'errore minimo alla variante.

La selezione della variante ottimale v^* avviene mediante minimizzazione lessicografica sulla coppia $(Err(v),\ E(v))$, dove $E(v)$ rappresenta la stima energetica per l'invocazione:

$$v^* = \arg\min_{v \in V} \big(Err(v),\ E(v)\big) \tag{3.4.1}$$

In altri termini, tra tutte le varianti disponibili viene selezionata quella con il punteggio di errore minore; in caso di parità, viene preferita la variante con il consumo energetico stimato più basso. La stima $E(v)$ è calcolata dalla funzione ausiliaria `EnergyEstimate`(v, *warm*), che distingue tra esecuzioni a caldo (*warm start*) e a freddo (*cold start*):

$$E(v) = \begin{cases} \mathit{InvocationJoule}(v) & \text{se } \mathit{isWarm} = \textbf{true} \\ \mathit{ColdStartJoule}(v) + \mathit{InvocationJoule}(v) & \text{se } \mathit{isWarm} = \textbf{false} \end{cases} \tag{3.4.2}$$

La determinazione dello stato warm viene effettuata dalla funzione `WarmHint`(v), che verifica l'esistenza di un container già attivo utilizzabile per la variante v. In particolare, se la variante espone il flag `ShareContainer = true`, il sistema considera warm anche i container di altre varianti appartenenti alla stessa funzione logica e con lo stesso runtime. Questo meccanismo di condivisione consente di ridurre l'impatto del cold start nelle situazioni in cui una variante priva di container attivo può beneficiare di un ambiente di esecuzione già inizializzato da una variante affine.

I valori *InvocationJoule* e *ColdStartJoule* utilizzati nella stima sono recuperati da *etcd* al momento della selezione e corrispondono all'ultima stima EMA disponibile per ciascuna variante, aggiornata asincronamente dal worker di raccolta descritto nella Sezione 3.3. Lo scheduler opera quindi sempre sulla stima più recente disponibile, con un ritardo rispetto al comportamento reale dell'ordine di grandezza del periodo del worker.

3.4.3 Selezione con vincolo energetico

Quando l'utente fornisce un budget energetico esplicito (`MaxEnergyJoule` > 0), lo scheduler adotta la politica di selezione con vincolo energetico, denominata *energy-budget aware*. In questa modalità il consumo energetico stimato diventa un vincolo

esplicito, e la minimizzazione dell'errore avviene all'interno dell'insieme delle varianti compatibili con tale vincolo.

Il processo di selezione si articola nelle fasi descritte di seguito.

Fase 1 — Partizionamento delle varianti. Sia $B =$ `MaxEnergyJoule` il budget dichiarato dall'utente. Tutte le varianti con stima energetica superiore a $B + \varepsilon$, dove $\varepsilon = 0.05 \cdot B$, vengono scartate a priori. Le rimanenti vengono ripartite in due insiemi disgiunti:

$$F_{\text{strict}} = \{\, v : E(v) \leq B \,\} \tag{3.4.3}$$

$$F_{\text{relaxed}} = \{\, v : B < E(v) \leq B + \varepsilon \,\} \tag{3.4.4}$$

F_{strict} contiene le varianti che rispettano il budget in modo esatto; F_{relaxed} quelle che lo superano entro una fascia del 5%, ammesse come zona di tolleranza. Se entrambi gli insiemi sono vuoti, l'invocazione viene rifiutata con HTTP 422.

Fase 2 — Identificazione dei candidati finali. Da ciascun insieme viene estratta la variante con errore minimo:

$$v_1 = \arg\min_{v \in F_{\text{strict}}} \big(Err(v),\ E(v)\big) \tag{3.4.5}$$

$$v_2 = \arg\min_{v \in F_{\text{relaxed}}} \big(Err(v),\ E(v)\big) \tag{3.4.6}$$

dove $Err(v)$ è il punteggio di errore della variante, definito nella Sezione 3.4.2, e il consumo energetico agisce solo a parità di errore. Se uno dei due insiemi è vuoto, il candidato corrispondente non esiste e la selezione finale è immediata.

Fase 3 — Confronto e selezione finale. Quando entrambi i candidati esistono, si pone la domanda: conviene sforare il budget scegliendo v_2? La risposta è

affermativa solo se il guadagno relativo in termini di riduzione dell'errore compensa proporzionalmente lo sforamento relativo rispetto al budget. La condizione è:

$$\frac{Err(v_1) - Err(v_2)}{Err(v_1)} > \frac{E(v_2) - B}{B} \tag{3.4.7}$$

Il lato sinistro esprime la riduzione relativa dell'errore ottenuta passando da v_1 a v_2, normalizzata rispetto all'errore di v_1 stesso — il punto di riferimento naturale, dato che è la migliore alternativa entro vincolo. Il lato destro esprime la percentuale di sforamento energetico rispetto al budget dichiarato, valore compreso per costruzione nell'intervallo (0, 0.05]. Se la condizione (3.4.7) è soddisfatta, si sceglie v_2; altrimenti, si sceglie v_1.

Si osservi che, quando $Err(v_1) = 0$, la scelta fra v_1 e v_2 è banalmente v_1 dato che rispetta il vincolo ed ha errore minimo.

Fase 4 — Gestione del fallimento. Se entrambi gli insiemi F_{strict} e F_{relaxed} sono vuoti — ovvero tutte le varianti superano $B + \varepsilon$ — l'invocazione viene rifiutata con HTTP 422, senza eseguire alcuna computazione.

La logica complessiva delle due politiche è formalizzata nell'Algoritmo 2.
L'implementazione dell'algoritmo riflette la scelta di mantenere separata la logica di selezione della variante dalla logica di assegnazione del container e di esecuzione: SelectEnergyAwareVariant restituisce esclusivamente la variante selezionata, lasciando invariato il percorso di esecuzione successivo.

Complessivamente, lo scheduler energy-aware introduce un livello di controllo sul trade-off tra consumo energetico e qualità del risultato che opera esclusivamente su base consensuale, rispetta i profili energetici aggiornati a runtime e si integra con il meccanismo di condivisione dei container tra varianti. Il risultato è un sistema adattivo che, pur rimanendo trasparente agli utenti che non desiderano intervenire sulla

selezione, è in grado di soddisfare vincoli energetici espliciti minimizzando l'errore del risultato compatibilmente con i vincoli imposti e, a parità di accuratezza, preferendo sempre la variante meno costosa energeticamente.

Algoritmo 2 `SelectEnergyAwareVariant`: politica a casi

```
1:  function SelectEnergyAwareVariant(r : *function.Request)
2:      base ← r.Fun
3:      V ← GetFunctionsByLogicalName(base.LogicalName)
4:      if V = ∅ then
5:          return (base, "fallback-base", nil)          ▷ nessuna variante registrata
6:      Eval ← []
7:      for fn ∈ V do
8:          warm ← HasWarmContainer(fn)
9:          E(fn) ← energyCostJoule(fn, warm)
10:         Err(fn) ← errorScore(fn)
11:         Eval.Append(fn, E(fn), Err(fn))
12:     if r.MaxEnergyJoule = nil or r.MaxEnergyJoule ≤ 0 then ▷ Case: error-first
13:         v* ← arg min_Eval(Err, E)
14:         return (v*, "error-first", nil)
15:     B ← r.MaxEnergyJoule                                 ▷ Caso B: energy-budget
16:     ε ← energyEpsilon(B)                                 ▷ ε = 0.05 · B
17:     F_strict ← {x ∈ Eval | x.E ≤ B}
18:     F_relaxed ← {x ∈ Eval | B < x.E ≤ B + ε}
19:     if F_strict = ∅ and F_relaxed = ∅ then
20:         return (nil, "no-feasible-variants", Error)      ▷ HTTP 422
21:     if F_strict = ∅ then
22:         v* ← arg min_F_relaxed(Err, E)
23:         return (v*, "relaxed-only", nil)
24:     if F_relaxed = ∅ then
25:         v* ← arg min_F_strict(Err, E)
26:         return (v*, "strict-only", nil)
27:     v1 ← arg min_F_strict(Err, E)
28:     v2 ← arg min_F_relaxed(Err, E)
29:     if (Err(v1) − Err(v2)) / Err(v1) ≥ (E(v2) − B) / B then
30:         return (v2, "relaxed-tradeoff", nil)
31:     else
32:         return (v1, "strict", nil)
```

Capitolo 4

Risultati Sperimentali

Capitolo 5

Conclusioni

Elenco delle figure

2.1 Esempio di domini di potenza definiti dal modello RAPL. Il dominio *Package* rappresenta il consumo energetico complessivo della CPU, mentre domini più specifici, come *PP0* e *DRAM*, forniscono stime riferite rispettivamente ai core di calcolo e alla memoria principale. 18

2.2 Architettura concettuale di PowerAPI. I contatori di prestazione hardware e le misure energetiche aggregate (RAPL) vengono combinati dal modello SmartWatts per stimare il consumo energetico dei singoli processi. 21

2.3 Architettura di Kepler. 22

2.4 Architettura del sistema Faashouse. Il controller integra un *Resource Scheduler* che, seguendo un ciclo di monitoraggio, analisi e pianificazione, decide il *placement* delle funzioni sui nodi worker in base allo stato energetico corrente. 27

2.5 Architettura di E^2FIS. I nodi edge sono caratterizzati da differenti capacità computazionali e profili di consumo. Il modulo di *Scheduling computation* determina periodicamente l'allocazione delle funzioni e il sottoinsieme di nodi da mantenere attivi. 29

2.6 Architettura di alto livello di GreenFaaS: monitoraggio degli endpoint, stima del costo energetico dei trasferimenti e scheduling energy-aware per l'allocazione dei task in ambienti federati eterogenei. 31

2.7 Schema architetturale del framework PARM. Il *Degradation Predictor* stima l'impatto del power capping sulle prestazioni, mentre il *Resource Re-allocator* adatta dinamicamente le decisioni di scaling in presenza di vincoli di potenza. 35

2.8 Schema concettuale di JouleGuard. Il sistema regola dinamicamente il livello di accuratezza dell'applicazione in funzione di un vincolo energetico esplicito, trattando il budget di potenza come parametro attivo nel processo di controllo. 37

2.9 Pipeline di traduzione automatica proposta in *Code Once, Run Green.* La trasformazione avviene in fase di build tramite un processo iterativo di conversione e validazione del codice. 37

3.1 Architettura di Kepler per la raccolta dei consumi energetici in ambienti bare-metal e virtuali. In ambienti bare-metal, Kepler accede direttamente ai contatori hardware tramite RAPL, ottenendo misurazioni real-time senza necessità di modelli predittivi. 44

3.2 Architettura dell'infrastruttura di monitoraggio energetico. I componenti di monitoraggio esterni (Kepler, Prometheus, Grafana) sono separati dagli store interni (etcd, InfluxDB), con Serverledge al centro che interagisce con entrambi i livelli. 45

3.3 Struttura delle chiavi etcd per la funzione logica `fibonacci` con tre varianti. Le chiavi sotto `/function/` contengono i profili completi aggiornati dal worker periodico. L'indice sotto `/index/logical/` consente allo scheduler di recuperare efficientemente le sole varianti della funzione richiesta, senza scorrere l'intero store. 53

3.4 Schema del feedback loop energetico. Il worker periodico legge le metriche cumulative da Kepler tramite Prometheus e il numero di invocazioni dal livello di esecuzione, calcola i delta, applica un filtro EMA e aggiorna il profilo persistito in etcd, archiviando contestualmente i campioni in InfluxDB. 55

Elenco dei codici

3.1 File di definizione delle varianti per la funzione `fibonacci`. Ciascuna variante specifica runtime, entry point, profilo energetico iniziale e modello di output. 48

3.2 File di definizione delle varianti per la funzione `ImageClassification`. Il campo `quality` esprime l'accuratezza in forma discreta, adatta a output non numericamente quantificabili. 50

List of Algorithms

1 `SubmitRequest` con variant selection opt-in 61

2 `SelectEnergyAwareVariant`: politica a casi 66

Bibliografia

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] E. van Eyk, L. Toader, S. Talluri, L. Versluis, A. Uta, and A. Iosup, "Serverless is more: From PaaS to present cloud computing," *IEEE Internet Computing*, vol. 22, no. 5, pp. 8–17, 2018.

[3] G. Russo, T. Mannucci, V. Cardellini, and F. Lo Presti, "Serverledge: Decentralized function-as-a-service for the edge-cloud continuum," in *2023 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, 2023, pp. 131–140.

[4] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2131–2165, 2021.

[5] B. Kar, W. Yahya, Y. Lin, and A. Ali, "Offloading using traditional optimization and machine learning in federated cloud–edge–fog systems: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1199–1226, 2023.

[6] M. Amaral *et al.*, "Kepler: A framework to calculate the energy consumption of containerized applications," in *Proceedings of IEEE*

CLOUD 2023, 2023. [Online]. Available: https://research.ibm.com/publications/kepler-a-framework-to-calculate-the-energy-consumption-of-containerized-applications

[7] H. Hoffmann, "Jouleguard: energy guarantees for approximate applications," in *Proceedings of the 25th Symposium on Operating Systems Principles*, ser. SOSP '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 198–214.

[8] M. Lipp, A. Kogler, D. Oswald, M. Schwarz, C. Easdon, C. Canella, and D. Gruss, "PLATYPUS: Software-based Power Side-Channel Attacks on x86," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021.

[9] G. Fieni, R. Rouvoy, and L. Seinturier, "SmartWatts: Self-Calibrating Software-Defined Power Meter for Containers," in *CCGRID 2020 - 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, Melbourne, Australia, May 2020. [Online]. Available: https://inria.hal.science/hal-02470128

[10] D. Freina, M. Jansen, and A. Trivedi, "A survey of energy measurement methodologies for computer systems," 2024.

[11] Intel Corporation, "Intel running average power limit (rapl)," https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html, 2023.

[12] V. Ostapenco, L. Lefèvre, A.-C. Orgerie, and B. Fichel, "Exploring rapl as a power capping leverage for power-constrained infrastructures," in *Algorithms and Architectures for Parallel Processing*, T. Zhu, J. Li, and A. Castiglione, Eds. Singapore: Springer Nature Singapore, 2025, pp. 323–333.

[13] M. Hähnel, B. Döbel, M. Völp, and H. Härtig, "Measuring energy consumption for short code paths using rapl," in *ACM SIGMETRICS Performance Evaluation Review*. New York, NY, USA: Association for Computing Machinery, 2012, p. 84–90. [Online]. Available: https://doi.org/10.1145/2425248.2425252

[14] PowerAPI Project. (2023) Hwpc sensor – powerapi documentation. [Online]. Available: https://powerapi.org/reference/overview/#usage_1

[15] ——. (2023) Smartwatts energy model. [Online]. Available: https://powerapi.org/reference/formulas/smartwatts/

[16] V. Shponarskyi, "Monitoring environmental impact of kubernetes clusters," https://medium.com/@vasyl.shponarskyi/monitoring-environmental-impact-of-kubernetes-clusters-8c81f72ac55f, 2022.

[17] Red Hat, "Introducing kepler: Efficient power monitoring for kubernetes," https://next.redhat.com/2023/08/22/introducing-kepler-efficient-power-monitoring-for-kubernetes/, 2023.

[18] M. S. Aslanpour, A. N. Toosi, M. A. Cheema, and M. B. Chhetri, "Faashouse: Sustainable serverless edge computing through energy-aware resource scheduling," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1533–1547, 2024.

[19] F. Righetti, B. Cornacchia, G. R. Russo, N. Tonellotto, V. Cardellini, and C. Vallati, "Energy-efficient function invocation scheduling for edge faas platforms," in *2025 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2025, pp. 18–25.

[20] A. Kamatar, V. Hayot-Sasson, Y. Babuji, A. Bauer, G. Rattihalli, N. Hogade, D. Milojicic, K. Chard, and I. Foster, "Greenfaas: Maximizing

energy efficiency of hpc workloads with faas," 2024. [Online]. Available: https://arxiv.org/abs/2406.17710

[21] T. Ciesielczyk, A. Cabrera, A. Oleksiak, W. Piątek, G. Waligóra, F. Almeida, and V. Blanco, "An approach to reduce energy consumption and performance losses on heterogeneous servers using power capping," *Journal of Scheduling*, vol. 24, no. 5, pp. 489–505, 2021.

[22] H. Qiu, L. Zhang, C. Wang, H. Franke, Z. T. Kalbarczyk, and R. K. Iyer, "Parm: Adaptive resource allocation for datacenter power capping," in *Machine Learning for Systems Workshop at NeurIPS*, 2023.

[23] S. Werner, M. Kähler, and A. Hakamian, "Code once, run green: Automated green code translation in serverless computing," in *2025 IEEE International Conference on Cloud Engineering (IC2E)*, 2025, pp. 105–113.

[24] M. Amaral, S. Choochotkaew, E. K. Lee, H. Chen, and T. Eilam, "Exploring Kepler's potentials: unveiling cloud application power consumption," 2023, cNCF Blog. [Online]. Available: https://www.cncf.io/blog/2023/10/11/exploring-keplers-potentials-unveiling-cloud-application-power-consumption/